

Robotic Mobile Fulfillment Systems: Strategies for Pod Selection and Scheduling

Kerim U Kizil¹ , Chelliah Sriskandarajah¹  and Jon M Stauffer¹ 

Production and Operations Management
2026, Vol. 35(2) 625–648
© The Author(s) 2025



Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/10591478251349902
journals.sagepub.com/home/pao



Abstract

Robotic mobile fulfillment (RMF) systems automate storage and transportation within fulfillment centers while still relying on human pickers. We focus on two key performance metrics in these systems, aiming to minimize overall completion time (OCT) as the primary objective and the number of required robots (NRR) as a secondary objective. We investigate two interrelated operational problems influencing these metrics: (i) Pod selection, which involves choosing the mobile racks for item picking, and (ii) pod scheduling, which entails assigning these racks to pickers and determining the picking sequence. We first explore the pod scheduling problem independently, providing theoretical results. This stand-alone problem is NP-hard with at least two pickers when minimizing OCT and remains NP-hard with even one picker when minimizing NRR. The NRR objective introduces a novel optimization problem structure, contributing to scheduling theory even beyond the RMF context. We demonstrate that a simple but effective scheduling rule is asymptotically optimal for minimizing OCT with multiple pickers. For NRR minimization with a single picker, we derive theoretical performance bounds for two sequencing rules. When incorporating the pod selection problem, we focus on two approaches: (i) Sequential and (ii) integrated pod selection and scheduling. The sequential approach handles pod selection first, then pod scheduling, while the integrated approach addresses both problems simultaneously within a single formulation to minimize OCT. Computational experiments using e-commerce order data reveal that the sequential approach achieves significantly faster and constant computation times, with a mean OCT similar to that of the integrated approach. We also find that our sequencing rules can reduce mean NRR by 13%–28% without affecting OCT, compared to the case where no such rules are used. The sequential approach for OCT minimization and the proposed sequencing rules for NRR minimization are scalable to large systems with any picker count. Our experiments also explore the changes in OCT and NRR values with varying numbers of pickers. These results guide managers in assigning the desired number of pickers for a target OCT and inform managers about the average NRR they can expect for the chosen picker count.

Keywords

Robotic mobile fulfillment, pod selection, scheduling, e-commerce, performance bounds

Date received 01 November 2024; accepted 06 May 2025 after one revision

Handling Editor: Kalyan Singhal

1 Introduction

As competition in e-commerce intensifies, firms must constantly improve operations to meet the rising customer expectations for fast and error-free deliveries. Order fulfillment has an increasingly critical role in today's competitive environment in this regard. A failure or delay in order fulfillment operations can lead to the loss of loyal customers and result in significant financial losses (Rao et al., 2011). Recognizing this reality, companies continuously seek innovative ways to improve fulfillment performance while reducing the associated costs. Robotics advancements draw significant attention

from e-commerce companies and logistics experts alike, as the automation of key fulfillment processes, such as storing, transporting, picking, and packing, offers increased efficiency, accuracy, and scalability.

¹Department of Information and Operations Management, Mays Business School, Texas A&M University, College Station, TX, USA

Corresponding author:

Chelliah Sriskandarajah, Department of Information and Operations Management, Mays Business School, Texas A&M University, USA.
Email: chelliah@mays.tamu.edu

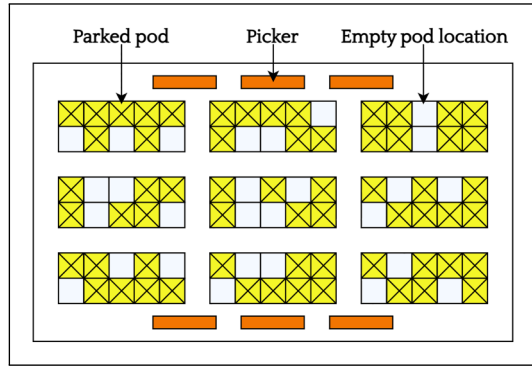


Figure 1. Layout of an example robotic mobile fulfillment (RMF) system, with pickers positioned on two sides.

Among various implementations of warehouse automation, robotic mobile fulfillment (RMF) stands out as a noteworthy and increasingly popular application. RMF automates the storing and transporting processes in a fulfillment center while leaving the picking operations to human employees. A typical RMF system is a part-to-picker order-picking system consisting of three key components: (i) Pickers, (ii) mobile racks (pods), and (iii) robots. Pickers are human employees who remain in their workstations throughout their shifts to pick items from pods that are positioned in front of them by robots. Pods are multi-level racks with several compartments to store items of the same or different stock keeping units (SKUs). They are designed to be transported by robots without human intervention. Robots travel on the fulfillment center floor, and when a pod needs to be moved, they get under the pod, lift it off the ground, and move the pod to a programmed destination. A pod can be moved by any robot in the system; therefore, RMF systems typically require fewer robots than pods. Pods in an RMF system are parked in a pod storage area while not in use (Figure 1). RMF systems usually provide high picking accuracies, often exceeding 99%.¹

RMF is preferred in fulfillment centers where items in stock are relatively small in size (e.g., small electronics, apparel, cosmetics), and a high number of orders are continuously fulfilled (Allgor et al., 2023). These centers often have multiple independent or loosely connected RMF systems, such as Amazon's five-story fulfillment center in Detroit (Reindl, 2023). This research focuses on one such RMF system and examines two critical operational problems: (i) Pod selection and (ii) pod scheduling.

As is common in practice, we consider an e-commerce company employing an order management system to retrieve orders from a backlog and allocate them to the focal RMF system. The focal system is stocked according to a given inventory allocation scheme, with a specific quantity of each SKU in each pod. A pod can contain items of different SKUs, and the inventory of an SKU can be distributed across several pods.² The company's order management system releases batches of orders to the focal RMF system periodically. Once

orders are released to the system, a *picking interval* is initiated, during which the system must complete the released orders without interruption. Given the orders at the beginning of each interval and the up-to-date inventory allocations, the RMF system must

- (i) select the pods to send to pickers and decide on pick quantities (pod selection problem) and
- (ii) schedule these pods with the pickers (pod scheduling problem).

The pod selection problem involves selecting the pods to process during a picking interval and determining the number of items of each SKU to pick from each selected pod to fulfill the orders assigned to the interval. Although items within a single customer order can be picked by different pickers at different times in the same interval and merged into complete orders in a separate workstation,³ the task of selecting the pods is still not straightforward as the inventory of an SKU may be distributed across multiple pods in the system. After selecting the pods to process, the pod scheduling problem

- (i) assigns a picker to process a selected pod (pod-to-picker assignment component) and
- (ii) determines the order in which the picker processes the pods (pod sequencing component).

We consider *planning horizons* (or simply, *horizons*) consisting of multiple consecutive picking intervals and evaluate two metrics as key performance indicators. First, the overall completion time (OCT) metric refers to the finish time of a planning horizon, i.e., the time point at which all orders assigned to the intervals within the horizon are fulfilled. Because the orders to be fulfilled are exogenous and given by the order management system, minimizing OCT effectively maximizes the order throughput rate for the planning horizon. The second metric of interest is the number of required robots (NRR) to seamlessly support the picking operations of a planning horizon.

We first address the pod scheduling problem (Figure 2), initially treating the pod selection solutions as given. This enables an independent investigation of pod scheduling to derive theoretical results for both OCT and NRR minimization. Then, we incorporate the pod selection problem into our study, considering the same two metrics. To minimize OCT, we propose two approaches: (i) Sequential pod selection and scheduling (SPS) and (ii) integrated pod selection and scheduling (IPS). The sequential approach addresses pod selection first, followed by pod scheduling, while the integrated method tackles both problems simultaneously within a single formulation to minimize OCT. Finally, we focus on the pod sequencing component of the pod scheduling problem and re-sequence the interval schedules obtained in the previous step with the aim of minimizing NRR while preserving the pod-to-picker assignments obtained previously to maintain the lowest OCT (Figure 2).

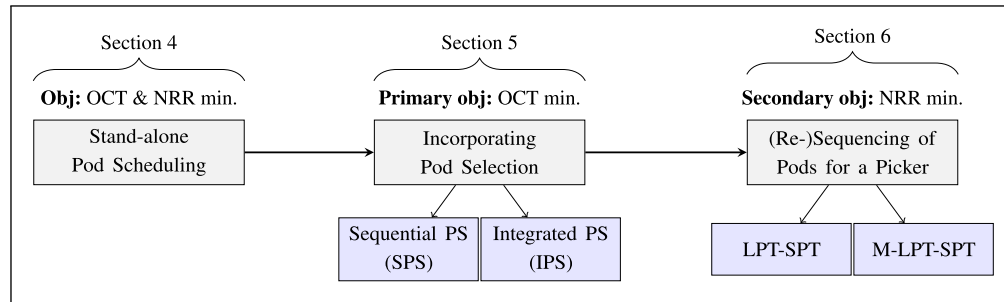


Figure 2. The study flow diagram.

Our theoretical contribution to the RMF literature can be summarized in three key points. First, to the best of our knowledge, ours is the first study to address both pod selection and pod scheduling problems in RMF with OCT and NRR minimization objectives. Our focus on these two objectives leads to two distinct problem structures for the stand-alone pod scheduling problem. Notably, the NRR minimization objective leads to a novel optimization problem structure, contributing to scheduling theory even beyond the RMF context. Second, we demonstrate that the stand-alone pod scheduling problem is NP-hard under both objectives. This problem is NP-hard with at least two pickers when minimizing OCT, and remains NP-hard with one picker when minimizing NRR. Third, we demonstrate that the *longest processing time first* (LPT) scheduling rule is asymptotically optimal for minimizing OCT with multiple pickers. For NRR minimization with a single picker, we derive theoretical performance bounds for the *longest/shortest processing time first* (LPT-SPT) sequencing rule. Additionally, we introduce a modified LPT-SPT sequencing rule (M-LPT-SPT), which performs better when a high proportion of pod processing times are short. This modified rule maintains the same theoretical performance bound as LPT-SPT.⁴

Our computational experiments using e-commerce order data reveal several managerial insights, three of which we highlight here. First, SPS maintains a consistently low computation time when minimizing OCT, regardless of the number of pickers in the system, whereas IPS requires longer computation times as the number of pickers increases. SPS is up to 7 times faster than IPS, and the mean planning horizon OCT for SPS is within half a percent of IPS across all considered instances. This finding demonstrates that the sequential pod selection and scheduling approach is preferred over the integrated approach (IPS) in practice due to its efficient computation time and scalability, with only a negligible impact on mean OCT.

Second, given the best OCT level found by the SPS approach, we apply LPT-SPT and M-LPT-SPT sequencing rules to resequence the pods for each picker to minimize NRR without changing the pod-to-picker assignments (last step in Figure 2). When comparing the sequencing rules for NRR minimization, we find there is no apparent difference between the two rules if pod processing times are heavily

left-skewed toward shorter durations. However, as processing times become more symmetric, LPT-SPT performs better for systems with longer pod processing times, while M-LPT-SPT is the better choice for systems with shorter pod processing times. We demonstrate that choosing the right rule reduces the number of required robots by 13% to 28%, without affecting OCT, compared to not using any of these sequencing rules.

Third, our computational study provides guidance for managers on the number of pickers needed in an RMF system to achieve a target mean OCT. While more pickers in the system can decrease OCT, it will inevitably increase NRR. We demonstrate that using one of our proposed sequencing rules can help RMF managers effectively address this trade-off. Our robustness and sensitivity analysis consisting of six extensions demonstrates that our managerial insights continue to hold under several changing system parameters and relaxed assumptions.

The remainder of the paper is organized as follows. Section 2 reviews the relevant literature, highlighting gaps addressed by our research. Section 3 introduces notation. Section 4 examines the pod scheduling problem independently to gain insights, while Section 5 incorporates the pod selection problem with an OCT minimization objective. Section 6 discusses the LPT-SPT and M-LPT-SPT sequencing rules for NRR minimization. Section 7 presents our computational study, and Section 8 concludes the paper. Proofs and additional details are provided in the electronic companion (EC), accessible online (doi: 10.1177/10591478251349902).

2 Literature Review

The complexity and variety of automated warehousing and fulfillment systems have only increased since their advent in the 1960s (De Koster, 2018), and today's logistics landscape involves numerous types of such systems deployed by businesses worldwide. Automated systems might utilize different technologies as their main driving force, such as automated cranes, forklifts, or autonomous vehicles. RMF is a relatively novel concept compared to the long history of warehouse automation practices (Azadeh et al., 2019; da Costa Barros and Nascimento, 2021). First operationalized by Kiva Systems in the 2000s (Guizzo, 2008), RMF systems are especially suited for e-commerce fulfillment centers where a diverse assortment

of relatively small-sized items are present, and high volumes of orders need to be fulfilled continuously (Rim  l   et al., 2022). In this section, we focus on the RMF literature and refer the interested reader to Azadeh et al. (2019) for a review of other types of automated warehousing systems.

The managerial problems in the design, deployment, and operation of RMF systems can be classified into four main types. First, inbound inventory needs to be divided among the pods of the system, constituting the inventory allocation problem. The inventory allocation problem can consider different objectives, such as maximizing the throughput rate (Lamballais et al., 2020), maximizing similarities of SKUs in a pod (Kim et al., 2020; Xiang et al., 2018; Yuan et al., 2021), minimizing the number of pod visits to stations (Xiang et al., 2018), and minimizing the total distance traveled by robots (Cezik et al., 2022; Yuan et al., 2021). This problem is beyond the scope of our research, but we present our results under two inventory allocation policies to demonstrate robustness. Second, given customer orders (consisting of either a single item or multiple items), pods to pick the items from need to be decided, which refers to the pod selection problem. Third, selected pods must be scheduled with specific pickers of the system, leading to a pod scheduling problem. As part of pod scheduling, a pod must be assigned to a picker (pod assignment), and the pod processing order for a picker must be determined (pod sequencing). Lastly, a fourth problem concerns the locations of pods in the storage area under the objectives of total pod distance minimization (e.g., Weidinger et al., 2018), total robot distance minimization (e.g., Yuan et al., 2019), and throughput maximization (e.g., Lamballais et al., 2017; Roy et al., 2019). Our study focuses on the second and third problems, i.e., pod selection and pod scheduling.

Table 1 summarizes selected studies from the RMF literature across multiple dimensions. To our knowledge, our paper is one of only two studies that consider both pod selection and pod scheduling problems in the same framework, the other being Boysen et al. (2017). In their case, they consider a single picker who can handle a limited number of customer orders simultaneously. In contrast, we focus on the case where pickers do not consolidate items into complete orders. We recognize that both approaches are used in real-world systems, and each creates a unique problem setting that warrants its own research focus. The metrics in our study—overall completion time and number of required robot minimization—also differ from that in Boysen et al. (2017), which focus on minimizing the number of pods utilized in a time window. We emphasize that our research directly models the number of robots required to support schedules seamlessly. This is a metric not previously investigated theoretically in the literature and leads to a novel optimization problem structure.

Next, we briefly discuss the three studies in Table 1 that investigate pod selection or scheduling problems. Note that all studies reviewed here involve a single RMF system, as does our paper. In an RMF system, pickers continuously pick items

from pods and put them into bins, which are then placed on conveyors. As such, picking operations constitute an integral part of such a system, and it is imperative to improve this step to ensure overall fulfillment performance. In this context, Xie et al. (2021) focus on the assignment of orders and pods to pickers, which we call the *pod-to-picker assignment* component of pod scheduling (the other component is pod sequencing). They use integer programming formulations to model pod-to-picker assignments and order-to-picker assignments. The authors find that the decomposition of orders into individual items can increase throughput. This finding partly explains why the industry often uses the strategy of splitting orders into individual items, allowing them to be picked by different pickers and/or at different times, as in our case. Wang et al. (2022) explore the human factor side of pod scheduling in RMF systems. The authors conclude that approaches that schedule pods with pickers should take human working state fluctuations into account to yield better performances. In our research, we consider human factors as an extension, where we account for the impact of human fatigue as an interval progresses. Boysen et al. (2017) determine the processing sequence of orders and arrival sequence of pods to pickers using dynamic programming and a heuristic beam search procedure. The authors provide analytical discussions on the number of pod moves objective. To the best of our knowledge, our study is the first to establish a direct theoretical relationship between pod schedules and the number of required robots to support them (cf. theoretical objectives column in Table 1). In particular, we establish the complexity status of pod scheduling as a stand-alone problem under the objectives of OCT and NRR.

Finally, we draw the reader's attention to the sizes of considered representative systems in the literature, measured in the number of pickers (#*P* column of Table 1). For example, Boysen et al. (2017) and Roy et al. (2019) simulate a system with a single picker in their respective studies. Conversely, Weidinger et al. (2018) consider a system as large as 20 pickers in some of their simulation experiments. We present a theoretical analysis of the stand-alone pod scheduling problem under the throughput maximization (i.e., OCT minimization) objective, focusing on two pickers for simplicity. Notably, the theorems established in Section 4.1 for this problem continue to hold if there are more than two pickers in the system. In a similar vein, the stand-alone pod scheduling problem of minimizing NRR is theoretically investigated with one picker in the system. In our numerical experiments with multiple pickers, we plan the number of robots for each individual picker. Hence, our theoretical results for NRR minimization continue to hold for the multi-picker case. In particular, we perform computational experiments to illustrate our approach up to 12 pickers, and it could handle many more. Lastly, we would like to refer interested readers to Allgor et al. (2023) and Qin et al. (2022) for details on RMF system deployments by Amazon and JD.com, respectively.

Table 1. Related RMF literature and position of this study.

Reference	(1)	(2)	(3)	(4)	CR	#P	Decisions	Theoretical Objectives	Methodologies
Boysen et al. (2017)	✓	✓	✓	✓	✓	1	Grouping orders to be fulfilled together; order sequences; pod schedules	Minimize # of pods utilized in a time window	Combinatorial opt.; heuristics; simulation
Lamballais et al. (2017)				✓		5	Layout of the storage area; dividing a system's storage area into zones	Maximize order throughput; minimize average order cycle time	Queuing theory; simulation
Weidinger et al. (2018)				✓	✓	20	Pod locations in storage area	Minimize total distance traveled by pods	Combinatorial opt.; heuristics
Xiang et al. (2018)	✓	✓				–	Inventory allocation to pods, grouping orders to be fulfilled together	Maximize similarities of SKUs in a pod; minimize # of pod visits to stations	Combinatorial opt.; heuristics
Roy et al. (2019)				✓		1	Dividing a system's storage area into zones; use of dedicated robots	Maximize throughput rates	Queuing theory; simulation
Yuan et al. (2019)	✓			✓		10	Pod locations in storage area	Minimize total distance traveled by robots	Continuous fluid model; simulation
Kim et al. (2020)	✓					–	Inventory allocation to pods	Maximize similarities of SKUs in a pod	Combinatorial opt.; heuristics
Lamballais et al. (2020)	✓					6	# of pods per SKU; # of stations; replenishment levels	Minimize order throughput time	Queuing theory; simulation
Xie et al. (2021)		✓			✓	4	Assignment of orders and pods to stations	Minimize # of pods utilized in a horizon; minimize idle capacity at stations	Combinatorial opt.; heuristics; simulation
Yuan et al. (2021)	✓			✓		6	Inventory allocation to pods; pod locations in storage area	Maximize similarities of SKUs in a pod; minimize total distance traveled by robots	Combinatorial opt.; heuristics; simulation
Cezik et al. (2022)	✓					–	Pods to replenish with received inventory, # of velocity classes	Minimize total distance traveled by robots	Continuous fluid model; simulation
Wang et al. (2022)			✓		✓	9	Pod schedules; robot schedules	Minimize expected total picking time considering human working states	Combinatorial opt.; dynamic programming
Our paper		✓	✓	✓	✓	12	Pod selection for picking; pod schedules (incl. pod-to-picker assignments and pod sequences)	Minimize overall completion time; minimize # of required robots	Combinatorial opt.; complexity theory; heuristics

Note. (1) Inventory Allocation; (2) Pod Selection; (3) Pod Scheduling; (4) Pod Location.

CR = computational complexity results; #P = maximum number of pickers considered; SKU = stock keeping unit; RMF = robotic mobile fulfillment.

References are listed in chronological order, with alphabetical ordering within the same year.

Table 2. Notation used throughout our study.

Sets	Description
$\mathcal{J}$	The set of SKUs in the RMF system, $\mathcal{J} = \{1, 2, \dots, I\}$
$\mathcal{D}$	The set of pods in the system, $\mathcal{D} = \{1, 2, \dots, D\}$
$\mathcal{D}(t)$	The set of pods processed in interval t , $\mathcal{D}(t) \subseteq \mathcal{D}$
$\mathcal{T}$	Planning horizon, i.e., a set of consecutive picking intervals, $\mathcal{T} = \{1, 2, \dots, T\}$
η^t	Defined as $\eta^t = \{\eta_d^t \mid d \in \mathcal{D}(t)\}$, where η_d^t is defined below as a scalar
$\mathcal{P}$	The set of pickers in the system, $\mathcal{P} = \{1, 2, \dots, k\}$
Scalars	Description
k	The number of pickers in the system, i.e., $ \mathcal{P} = k$
m_i^t	The number of items to pick of SKU i in interval t
Λ_{id}^t	The number of items of SKU i present in pod d at the start of interval t
ϕ_d^t	The number of items to be picked from pod d in interval t
α	Unit item picking time
β	Pod setup time
η_d^t	Processing time of pod d in interval t , defined as $\eta_d^t = \beta + \alpha \times \phi_d^t$
$\tau^t(\pi^t, d)$	The time point in schedule π^t when the picking operation of pod d starts
$\tau''(\pi^t, d)$	The time point in schedule π^t when the picking operation of pod d ends
γ	One-way robot travel time between pod storage area and a picker of the system
c	Pod capacity (in number of items)
Functions	Description
$f_{OCT}(\cdot)$	Returns the overall completion time of a given schedule
$f_{NRR}(\cdot)$	Returns the number of required robots to support a given schedule
Schedules	Description
σ_j^t	The pod picking sequence for picker j in interval t
π^t	The schedule for interval t , where $\pi^t = (\sigma_1^t, \sigma_2^t, \dots, \sigma_k^t)$
$\bar{\pi}$	The horizon schedule for a planning horizon $\mathcal{T}$, where $\bar{\pi} = [\pi^1, \pi^2, \dots, \pi^T]$
Problems	Description
PodSched-OCT(k)	The pod scheduling problem minimizing OCT with k pickers
PodSched-NRR(k)	The pod scheduling problem minimizing NRR with k pickers

Note. This table does not include symbols exclusively used in presenting mathematical formulations. The reader can refer to the paragraphs prior to mathematical formulations for details on formulation-specific notation. RMF = robotic mobile fulfillment; OCT = overall completion time; NRR = number of required robots; SKU = stock keeping unit.

3 Notation

We consider an RMF system operating continuously across multiple picking intervals. SKUs in the system constitute the set $\mathcal{J}$, while the pods of the system form the set $\mathcal{D}$. Fulfillment operations in the system are organized into multiple picking intervals, which collectively constitute the planning horizon $\mathcal{T}$. In an interval t , the system is responsible for fulfilling orders pulled from a backlog (Allgor et al., 2023) by meeting the associated item-picking demands. Hence, an interval $t \in \mathcal{T}$ can be described by a set of two-tuples $\{(i, m_i^t) \mid i \in \mathcal{J}\}$, where i denotes an SKU and m_i^t is the number of items required to be picked of SKU i to fulfill every order assigned to interval t . The notation used in this paper is summarized in Table 2, while Table 3 provides a list of abbreviations for quick reference.

This study builds on two practice-supported assumptions. First, we treat multi-item orders as fully decomposable at the

individual item level at the picking step. Thus, pickers solely focus on retrieving items from pods, placing them in labeled bins, and placing those bins on conveyors. As noted in practical application literature (Allgor et al., 2023; Xie et al., 2021; Yuan et al., 2019), order merging and packing operations are conducted in separate stations and outside the picking scope. As such, merging and packing are beyond the scope of our research. Second, again in line with literature and practice, item-picking demands are known at the start of an interval, and an order is only assigned to a particular RMF system if there is sufficient inventory to fulfill it at the start of an interval. However, we relax this second assumption when we consider stockouts in our computational extensions. Having introduced the fundamental notation and the setting, we now present the pod scheduling problem independent of the pod selection problem (i.e., taking the pod selection solution as given).

Table 3. Key abbreviations and their descriptions.

Abbreviation	Description
IPS	Integrated pod selection and scheduling
LPT	Longest processing time first rule
LPT-SPT	Longest Processing time-shortest processing time first rule
M-LPT-SPT	Modified longest processing time-shortest processing time first rule
NRR	Number of required robots
OCT	Overall completion time
RMF	Robotic mobile fulfillment
SKU	Stock keeping unit
SPS	Sequential pod selection and scheduling

4 Pod Scheduling

Defining and investigating the pod scheduling problem independent of the pod selection problem can provide theoretical insights into its structure. Furthermore, the solution to the stand-alone pod scheduling problem can be applied in any sequential approach that handles pod selection and scheduling separately. We build such a sequential approach in Section 5.1, which is later shown to achieve faster computation times compared to an integrated approach while still producing comparable solutions. To this end, we next explore the stand-alone pod scheduling problem under the OCT and NRR minimization objectives, respectively.

The pod scheduling problem must be solved at the beginning of each interval. Assume that subsequent to the solution of the pod selection problem for an interval t , the number of items to pick from each pod is known, denoted $\phi_d^t \in \mathbb{Z}_0^+$, where $\mathbb{Z}_0^+$ is the set of non-negative integers. The goal of the pod scheduling problem is to use this information to optimize the system metrics. For a given interval t , any solution to a scheduling problem is denoted $\pi^t = (\sigma_1^t, \sigma_2^t, \dots, \sigma_k^t)$ and called a *schedule*, where σ_j^t for $j = 1, \dots, k$ are the pod sequences for the k pickers in interval t . As noted in Table 2, we call T consecutive picking intervals a *planning horizon*. Similarly, the concatenation of T schedules for a planning horizon $\mathcal{T}$ is called a *horizon schedule*, and denoted with $\bar{\pi} = [\pi^1, \pi^2, \dots, \pi^T]$.

4.1 Pod Scheduling Problem Under OCT Minimization

We use the notation $\text{PodSched-OCT}(k)$ to denote the stand-alone pod scheduling problem in an interval t under the OCT minimization objective with k pickers in the system. Because this problem only concerns OCT minimization, we assume sufficiently many robots in the system to support seamless picking operations. This assumption is subsequently relaxed in Section 4.2 by focusing on the pod scheduling problem with NRR minimization. To facilitate our theoretical work, we assume a picker takes $\alpha \in \mathbb{Z}^+$ minutes to retrieve a single item

from a pod and put it in a bin with an identifier of the customer order. Similarly, starting the picking operations of a pod takes $\beta \in \mathbb{Z}^+$ minutes for setup. The amount β is not—and does not include—the traveling time of the pods between the storage area and pickers and only represents the time it takes a picker to get ready to process a pod. For example, upon a pod's arrival, a picker needs to gather empty bins to contain the picked items, read the pick list, inspect the pod visually to confirm its contents match the list, and set up the handheld scanner used while picking items. We refer to α and β as picking time (for an item) and setup time (for a pod), respectively. The time $\eta_d^t = \beta + \alpha \times \phi_d^t$ is then the *processing time* of pod d in interval t .⁵ We also define a subset of pods here, $\mathcal{D}(t) := \{d \mid d \in \mathcal{D}, \phi_d^t > 0\}$, to denote the pods processed in interval t , where $\mathcal{D}(t) \subseteq \mathcal{D}$ and $D(t) := |\mathcal{D}(t)|$. We use $\tau'(\pi^t, d)$ [respectively, $\tau''(\pi^t, d)$] to indicate the time point in schedule π^t when the picking operation of pod d starts [respectively, finishes]. We formally define $\text{PodSched-OCT}(k)$ as follows.

DEFINITION 1. Consider an RMF system with k pickers and a single interval t . Then, given the set of pod processing times $\eta^t = \{\eta_d^t = \beta + \alpha \times \phi_d^t \mid d \in \mathcal{D}(t)\}$, $\text{PodSched-OCT}(k)$ is the problem of finding a schedule $\pi^t = (\sigma_1^t, \sigma_2^t, \dots, \sigma_k^t)$ for the picking operations of pods in $\mathcal{D}(t)$ such that $f_{\text{OCT}}(\eta^t; \pi^t) = \max\{\tau''(\pi^t, d) \mid d \in \mathcal{D}(t)\}$ is minimized.

The set η^t characterizes an instance of $\text{PodSched-OCT}(k)$. Note that $\text{PodSched-OCT}(k)$ is a static problem by definition, as it focuses on a single interval t . In what follows, we first set $k = 2$ and explore the theoretical properties of $\text{PodSched-OCT}(2)$. Since customer orders are fully decomposable at the picking step, at least one optimal solution to $\text{PodSched-OCT}(2)$ will pick all ϕ_d^t items from pod d consecutively to avoid the occurrence of multiple pod setup times, β , for the same pod. To formalize this intuition as a lemma, we define the concept of a picking run. A picking run for pod $d \in \mathcal{D}(t)$ starts when a picker sets up to pick items from pod d and continues until the same picker sets up to pick items from the next pod $\tilde{d} \in \mathcal{D}(t)$, where $\tilde{d} \neq d$.

LEMMA 1. *Considering a single interval t , in at least one optimal solution to $\text{PodSched-OCT}(2)$, each pod in $\mathcal{D}(t)$ will be picked in a single picking run.*

The proof of this lemma is provided in Section EC.1 in the e-companion. This lemma implies that $\text{PodSched-OCT}(2)$ can be solved by partitioning the set of pod processing times $\eta^t = \{\eta_d^t = \beta + \alpha \times \phi_d^t \mid d \in \mathcal{D}(t)\}$ into two mutually exclusive and collectively exhaustive subsets such that the sums of η_d^t values in two subsets are as close as possible to each other. Building on this implication, the theorem below establishes the complexity of $\text{PodSched-OCT}(2)$.

THEOREM 1. *PodSched-OCT(2), i.e., the pod scheduling problem under the OCT minimization objective for a picking interval t and two pickers in the system, is NP-hard.*

The proof of this theorem is given in Section EC.1. Although PodSched-OCT(2) is NP-hard, a simple scheduling rule, LPT, provides asymptotically optimal solutions in minimizing OCT. The LPT rule orders the pods in decreasing pod processing times and assigns them in this order to the earliest available picker. We formalize the asymptotical optimality of this rule with the following theorem.

THEOREM 2. *The LPT rule provides an asymptotically optimal solution for PodSched-OCT(2), i.e., the pod scheduling problem under the OCT minimization objective for a picking interval t and two pickers in the system.*

The proof of this theorem is given in Section EC.1. Note that the results in Theorems 1 and 2 hold true for the general problem of PodSched-OCT(k) where $k > 2$. The asymptotic optimality of the LPT rule for PodSched-OCT(k) implies RMF systems can effectively reduce the OCT of an interval by processing pods with longer processing times earlier in the interval. However, when the NRR minimization objective is considered, LPT has an inherent drawback. With the LPT rule, pods with the shortest pod processing times are scheduled at the end of an interval. Hence, to ensure seamless picking operations towards the end of an interval, many pods need to travel to pickers in a very short timeframe, each pod requiring a robot for transportation. This, in turn, leads to a sharp increase in NRR. Our analysis of the pod scheduling problem under the NRR minimization objective leads us to consider another rule, LPT-SPT. This rule schedules the pod with the longest processing time as the first pod, followed by the shortest, and then alternates between the next longest and shortest until all pods are scheduled. The LPT-SPT rule will be used as a sequencing rule to resequence the pods assigned to a picker by any method.

We also observe that the structure of the pod scheduling problem under OCT minimization resembles the makespan minimization problem with identical parallel machines (Graham, 1969). However, this structural similarity disappears entirely when the pod selection problem is incorporated in Section 5. In contrast, the pod scheduling problem under NRR minimization (when the pod selection problem is solved a priori) has a novel and unique structure that, to the best of our knowledge, has not been previously investigated in any stream of literature, including the machine scheduling literature.

For a given picking interval t , considering different picker-to-pod ratios, $k/D(t)$, can provide additional insights into the theoretical analysis of PodSched-OCT(k). Leveraging the similarity between our stand-alone pod scheduling problem (under OCT minimization) and makespan minimization with identical parallel machines, we note two specific cases:

- **Case I:** This case arises when $k/D(t) \geq 1$ for a given interval t . In this scenario, the optimal solution to PodSched-OCT(k) is trivial and can be constructed by assigning each picker at most one pod. We further analyze this case under both OCT and NRR objectives in Section 4.3, referring to it as *Restricted Form A* therein.

- **Case II:** This case applies when

- (a) $1 > k/D(t) \geq 0.5$ and

- (b) the optimal solution to PodSched-OCT(k) contains at most two pods per picker.

In this case, the LPT rule is optimal in solving PodSched-OCT(k), following from Pinedo (2012: p. 114). This observation constitutes a stronger theoretical result than the rule's asymptotic optimality presented in Theorem 2 but has limited practical applicability because it requires partially knowing the structure of the optimal schedule (i.e., whether it contains at most two pods per picker).

Regardless of whether either of these two cases holds, a useful worst-case performance bound on the LPT rule can be established. Consider a general instance of the problem PodSched-OCT(k) for an interval t , denoted η^t . Let π_L^t be an LPT schedule for this instance, expressed as $\pi_L^t = (\sigma_{1,L}^t, \sigma_{2,L}^t, \dots, \sigma_{k,L}^t)$, where $\sigma_{p,L}^t$ represents the picking sequence of picker p . Define $\omega := \min_{p=1}^k |\sigma_{p,L}^t|$ as the minimum number of pods assigned to any picker in π_L^t . Let π_*^t denote the optimal schedule for the instance η^t . Then, the below worst-case bound follows from Coffman and Sethi (1976).

$$\frac{f_{OCT}(\eta^t; \pi_L^t)}{f_{OCT}(\eta^t; \pi_*^t)} \leq \frac{\omega + 1}{\omega} - \frac{1}{k\omega} \quad (1)$$

Fixing the number of pickers (such that $k \geq 2$), the bound in relation (1) decreases as the minimum number of pods assigned to a picker (ω) increases. Consequently, as the system size grows—measured by the number of pods each picker processes—the LPT rule is expected to perform closer to optimal in solving PodSched-OCT(k). This aligns with the rule's asymptotic optimality result in Theorem 2 but provides a more precise assessment of how the rule will perform for a given system in the worst case, as relation (1) allows for computing a worst-case bound for any given system size.

Lastly, we briefly discuss how picker-to-pod ratios can help explain the system's response to different pod capacity values (c), measured as the number of items a pod can hold. Specifically, when the number of pickers (k) is fixed, there exists a pod capacity threshold $c' \in \mathbb{Z}^+$ such that if $c \geq c'$, then $k/D(t) \geq 1$ for all t , and Case I applies. This implies that c is sufficiently large (at least c') for the demand in each picking interval to be met with k or fewer pods—an unlikely scenario in practice. Therefore, we focus on the case where $c < c'$ and $k/D(t) < 1$. In this scenario, insights can be drawn from the bound in relation (1). As c increases, the minimum number of pods assigned to a picker (ω) is expected to decrease since fewer pods are required to fulfill the same

demand. Consequently, the worst-case bound would increase, causing the LPT rule to potentially deviate further from optimality in solving PodSched-OCT(k). However, OCT is still expected to decrease as c increases, as fewer pods result in fewer instances of pod setup time (β). Acknowledging that this theoretical discussion is for the worst-case bound, we computationally examine the average realized impact of varying c values in Section 7.4.

4.2 Pod Scheduling Problem Under NRR Minimization

Similar to the case with OCT minimization, we use the notation PodSched-NRR(k) to denote the stand-alone pod scheduling problem, considering a single interval under the NRR minimization objective with k pickers in the system. To maintain theoretical tractability, we specify a constant one-way robot travel time of $\gamma \in \mathbb{Z}^+$ minutes between a picker and the pod storage area. In practice, γ can be chosen to represent the expected value of the travel time random variable under typical conditions. Also, substituting γ with maximum and minimum possible travel time can provide upper and lower bounds for the NRR bounds derived below. We relax the assumption of constant travel time in a computational extension in Section 7.4. Given the OCT schedule, we define PodSched-NRR(k) as follows without changing the pods assigned to a picker (i.e., without affecting the OCT value).

DEFINITION 2. Consider an RMF system with k pickers and a single interval t . Then, given the set of pod processing times $\eta^t = \{\eta_d^t = \beta + \alpha \times \phi_d^t \mid d \in \mathcal{D}(t)\}$ and a one-way robot traveling time $\gamma \in \mathbb{Z}^+$, PodSched-NRR(k) is the problem of finding a schedule $\pi^t = (\sigma_1^t, \sigma_2^t, \dots, \sigma_k^t)$ for the picking operations of pods in $\mathcal{D}(t)$ such that no picker is idle and the number of required robots $f_{NRR}(\eta^t; \pi^t)$ to support π^t is minimized without changing the pod-to-picker assignments.

The set η^t and the value of γ characterize an instance of PodSched-NRR(k). In what follows, we focus on the case with $k = 1$ picker for analytical tractability. Our goal about PodSched-NRR(1) here is to (i) show the problem is NP-hard even considering a single interval and a single picker, and (ii) find a scheduling rule with proven theoretical bounds to address the problem over several intervals. The first step towards the complexity proof is to show that the decision version of PodSched-NRR(1) is in class $\mathcal{NP}$. Given any instance of PodSched-NRR(1) and a solution $\pi^t = (\sigma_1^t)$ for it, we can argue that the decision version is in class $\mathcal{NP}$ if $f_{NRR}(\eta^t; \pi^t)$ can be computed in polynomial time for a given solution π^t . To this end, we first show that computing $f_{NRR}(\eta^t; \pi^t)$ is equivalent to solving a maximum number of overlapping intervals problem (MNOIP) if a solution π^t is given.⁶ Given multiple real intervals, MNOIP finds the maximum number of overlapping real intervals, which might occur at any point.

Given any schedule π^t for a picking interval t , we can create a corresponding MNOIP instance as follows. The number of real intervals is equal to $|\mathcal{D}(t)| = D(t)$, each for a pod $d \in \mathcal{D}(t)$. Then, the real interval associated with a pod d is $(\tau'(\pi^t, d) - \gamma, \tau''(\pi^t, d) + \gamma)$, during which a robot must be dedicated to pod d . Specifically, γ minutes before the start of picking operations for pod d , a robot must begin moving the pod to a picker. The robot remains under the pod as the picker retrieves items, and afterward, it must return the pod to storage for an additional γ minutes. Consequently, the maximum number of overlapping real intervals among these $D(t)$ real intervals will determine the number of robots required to support schedule π^t . Consider the following example.

EXAMPLE 1 (Calculating NRR with MNOIP). For a given picking interval t , consider the following PodSched-NRR(1) instance. Let $\mathcal{D}(t) = \{1, 2, \dots, 8\}$. Furthermore, consider the following processing times for these eight pods: $\eta_1^t = \eta_4^t = \eta_7^t = 5$ minutes and $\eta_2^t = \eta_3^t = \eta_5^t = \eta_6^t = \eta_8^t = 2$ minutes. Say the one-way travel time is $\gamma = 3$ minutes. Given a schedule $\pi^t = (\sigma_1^t)$ where $\sigma_1^t = \{1, 2, 3, 4, 5, 6, 7, 8\}$, we can build the Gantt chart in Figure 3. In this figure, the horizontal lines represent the real intervals of the corresponding MNOIP instance. In particular, assuming the first pod starts processing at time point 0, these lines denote the following eight real intervals: $(-3, 8)$, $(2, 10)$, $(4, 12)$, $(6, 17)$, $(11, 19)$, $(13, 21)$, $(15, 26)$, and $(20, 28)$. Each real interval represents a period during which a robot is occupied with a pod by:

- (i) Moving it to a picker,
- (ii) remaining under it during item retrieval, and
- (iii) returning it to the storage area.

So, in the figure, the top horizontal line represents the time period during which a robot is occupied with pod $d = 1$, the second line from the top represents a robot's busy time for pod $d = 2$, and so forth. The maximum number of overlapping real intervals for this MNOIP instance is four, occurring in the highlighted areas in the figure. Thus, four robots will be required concurrently in these regions. Consequently, $f_{NRR}(\eta^t; \pi^t) = 4$ robots are required to support this schedule without any picker idle times.⁷

Since an $O(N \log N)$ algorithm exists to solve any given MNOIP instance (Section EC.2 in the e-companion), the decision version of PodSched-NRR(1) is in class $\mathcal{NP}$. However, this computational calculation of the NRR objective lacks the algebraic manipulability required for the complexity proof of PodSched-NRR(1). Therefore, we introduce a novel matrix algebra approach to calculate NRR from a given schedule, requiring additional notation and a defined distance measure.

DEFINITION 3. For PodSched-NRR(1) in a given interval t , let $\bar{d}$ and $\bar{\bar{d}}$ be two pods present in a given schedule $\pi^t = (\sigma_1^t)$. Let ∇ be the set of pods scheduled between pods $\bar{d}$

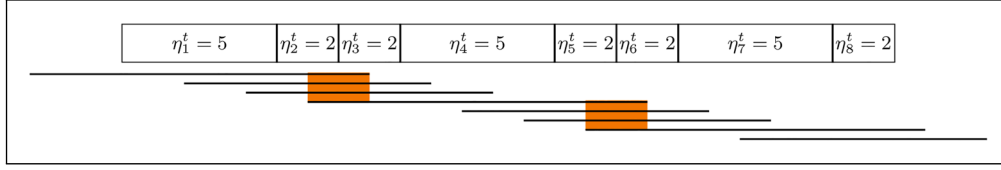


Figure 3. The schedule π^t for Example 1 with corresponding real intervals given as horizontal lines.

and $\bar{d}$ in sequence σ_1^t , excluding these two pods. Then, the distance between pods $\bar{d}$ and $\bar{d}$ in schedule π^t is defined as $\text{dist}(\pi^t; \bar{d}, \bar{d}) = \sum_{d \in \mathcal{V}} \eta_d^t$.

Definition 3 implies that if two pods are processed consecutively, $\nabla = \emptyset$ and the distance (in terms of time) between the two pods is zero. Conversely, if two pods are not consecutive in a schedule $\pi^t = (\sigma_1^t)$, then the distance between those pods is equal to the processing time sum of pods in between. Also, a pod's distance to itself is zero by this definition. In the below example, we build on Example 1 to demonstrate how this distance measure helps us calculate $f_{\text{NRR}}(\eta^t; \pi^t)$ for a given solution π^t to PodSched-NRR(1).

EXAMPLE 2 (Calculating NRR with Distance Matrices). Assume the same PodSched-NRR(1) instance and solution π^t as in Example 1. We use the distance definition to build a distance matrix for schedule π^t . Since distance matrices are symmetric (Definition 3), we can remove their lower triangular parts without losing information. For π^t (cf. Figure 3), we call the resulting matrix $M(\eta^t; \pi^t)$, as shown below.

$$M(\eta^t; \pi^t) = \begin{bmatrix} 0 & 0 & 2 & 4 & 9 & 11 & 13 & 18 \\ & 0 & 0 & 2 & 7 & 9 & 11 & 16 \\ & & 0 & 0 & 5 & 7 & 9 & 14 \\ & & & 0 & 0 & 2 & 4 & 9 \\ & & & & 0 & 0 & 2 & 7 \\ & & & & & 0 & 0 & 5 \\ & & & & & & 0 & 0 \\ & & & & & & & 0 \end{bmatrix} \quad (2)$$

Define $E(\eta^t; \pi^t) := M(\eta^t; \pi^t) - 2\gamma$. Since γ is the one-way travel time between the pod storage area and a picker, subtracting 2γ accounts for the additional time a robot spends moving the pod. With $\gamma = 3$ in our continuing example, the resulting matrix is as follows.

$$E(\eta^t; \pi^t) = \begin{bmatrix} -6 & -6 & -4 & -2 & 3 & 5 & 7 & 12 \\ & -6 & -6 & -4 & 1 & 3 & 5 & 10 \\ & & -6 & -6 & -1 & 1 & 3 & 8 \\ & & & -6 & -6 & -4 & -2 & 3 \\ & & & & -6 & -6 & -4 & 1 \\ & & & & & -6 & -6 & -1 \\ & & & & & & -6 & -6 \\ & & & & & & & -6 \end{bmatrix} \quad (3)$$

Finally, denoting the (i, j) th element of $E(\eta^t; \pi^t)$ with e_{ij} , we have $f_{\text{NRR}}(\eta^t; \pi^t) = \max_i \sum_{j=1}^{D(i)} \mathbb{1}(e_{ij} < 0)$, where $\mathbb{1}(\cdot)$ is the

indicator function returning 1 if the statement is true and 0 otherwise. In other words, the maximum number of negative elements in a row of $E(\eta^t; \pi^t)$ is the number of robots required by schedule π^t . In the continuing example, we have $f_{\text{NRR}}(\eta^t; \pi^t) = \max_i \sum_{j=1}^{D(i)} \mathbb{1}(e_{ij} < 0) = \{4, 3, 3, 4, 3, 3, 2, 1\} = 4$, which was also the result obtained by solving the corresponding MNOIP instance. This distance matrix approach to NRR calculation is helpful in establishing the complexity status of PodSched-NRR(1), presented below.

THEOREM 3. *PodSched-NRR(1), i.e., the pod scheduling problem under the NRR minimization objective for a picking interval t and one picker in the system, is NP-hard.*

The proof of Theorem 3, given in Section EC.1, is by reduction from the equal-cardinality partition problem, which is known to be NP-complete (Garey and Johnson, 1979: SP12 on p. 223). Although PodSched-NRR(1) is shown to be NP-hard even considering one interval, if we assume for simplicity that the same interval t is repeated T times during a planning horizon $\mathcal{T}$, then any list scheduling rule (i.e., any arbitrary order picking sequence rule) can be shown to have a certain performance bound for the planning horizon. To this end, we define this specific (repeating) version of PodSched-NRR(1), as follows.

DEFINITION 4. Consider an RMF system with one picker and a planning horizon $\mathcal{T}$ consisting of T intervals. The intervals have the same pod processing times, i.e., $\eta^t = \{\eta_d^t = \beta + \alpha \times \phi_d^t \mid d \in \mathcal{D}(t)\}$ for all $t \in T$ and $\eta = \{\eta^t \mid t \in \mathcal{T}\}$. Then, given a one-way traveling time $\gamma \in \mathbb{Z}^+$, Rep-PodSched-NRR(1) is the problem of finding a schedule $\pi^t = (\sigma_1^t)$ that will repeat throughout T intervals to form a horizon schedule $\tilde{\pi} = [\pi^1, \pi^2, \dots, \pi^T]$ while ensuring no picker is idle either in or between intervals and the number of required robots $f_{\text{NRR}}(\eta; \tilde{\pi})$ to support $\tilde{\pi}$ is minimized.

Let $\ell = \min(\eta)$, $m = \text{med}(\eta)$, and $u = \max(\eta)$ be the minimum, median, and maximum processing times in η , respectively. We denote the length of a picking interval with G . We assume that $\ell + u < 2\gamma < G$. In practice, these relationships hold as G is typically very large compared to 2γ (i.e., the two-way traveling time between storage area and pickers). Then, we can note the following performance bound for any list scheduling rule (i.e., any arbitrary order picking sequence rule) used to solve a Rep-PodSched-NRR(1) instance.

THEOREM 4. Consider an instance of *Rep-PodSched-NRR(1)* characterized by the set $\eta = \{\eta^t \mid t \in \mathcal{T}\}$ and a one-way travel time $\gamma \in \mathbb{Z}^+$. Let $\bar{\pi}_*$ be the optimal horizon schedule for this problem instance. Furthermore, let $\bar{\pi}_H$ be a horizon schedule obtained by any scheduling rule. Then, we have the following performance ratio bound.

$$\frac{f_{NRR}(\eta; \bar{\pi}_H)}{f_{NRR}(\eta; \bar{\pi}_*)} \leq \frac{\left\lceil \frac{2\gamma}{\ell} \right\rceil + 1}{\left\lceil \frac{2\gamma}{u} \right\rceil + 1} = R^a$$

The proof of Theorem 4 and the lemmas leading to it are given in Section EC.3. Theorem 4 sets a worst-case bound for the performance of any list scheduling rule. In the next two theorems and the resulting corollary, we show that the LPT-SPT rule has a better performance bound for the problem *Rep-PodSched-NRR(1)*. As the name suggests, for a given interval t , this rule builds schedule $\pi^t = (\sigma_1^t)$ by scheduling the pod with the longest processing time as the first pod, followed by the shortest, and then alternating between the next longest and shortest until all pods are sequenced.

THEOREM 5. Consider an instance of *Rep-PodSched-NRR(1)* characterized by the set $\eta = \{\eta^t \mid t \in \mathcal{T}\}$ and a one-way travel time $\gamma \in \mathbb{Z}^+$. Let $\bar{\pi}_*$ be the optimal horizon schedule for this problem instance. Furthermore, let $\bar{\pi}_{LS}$ be a horizon schedule obtained by the LPT-SPT rule. Then, we have the following performance ratio bound.

$$\frac{f_{NRR}(\eta; \bar{\pi}_{LS})}{f_{NRR}(\eta; \bar{\pi}_*)} \leq \frac{2 \left\lceil \frac{2\gamma}{m+\ell} \right\rceil + 1}{\left\lceil \frac{2\gamma}{u} \right\rceil + 1} = R'$$

The proof of Theorem 5 and the lemmas leading to it are given in Section EC.4. The below theorem establishes a second performance bound for the LPT-SPT rule, which then leads to a corollary.

THEOREM 6. Consider an instance of *Rep-PodSched-NRR(1)* characterized by the set $\eta = \{\eta^t \mid t \in \mathcal{T}\}$ and a one-way travel time $\gamma \in \mathbb{Z}^+$. Let $\bar{\pi}_*$ be the optimal horizon schedule for this problem instance. Furthermore, let $\bar{\pi}_{LS}$ be a horizon schedule obtained by the LPT-SPT rule. Then, we have the following performance ratio bound.

$$\frac{f_{NRR}(\eta; \bar{\pi}_{LS})}{f_{NRR}(\eta; \bar{\pi}_*)} \leq \frac{\left\lceil \frac{2\gamma}{\ell} \right\rceil + 1}{2 \left\lceil \frac{2\gamma}{u+m} \right\rceil} = R''$$

The proof of Theorem 6 and the lemmas leading to it are given in Section EC.5. The corollary follows.

COROLLARY 1. Consider an instance of *Rep-PodSched-NRR(1)* characterized by the set $\eta = \{\eta^t \mid t \in \mathcal{T}\}$ and a one-way travel time $\gamma \in \mathbb{Z}^+$. Let $\bar{\pi}_*$ be the optimal horizon

schedule for this problem instance. Furthermore, let $\bar{\pi}_{LS}$ be a horizon schedule obtained by the LPT-SPT rule. Then, we have the following performance ratio bound.

$$\frac{f_{NRR}(\eta; \bar{\pi}_{LS})}{f_{NRR}(\eta; \bar{\pi}_*)} \leq \min \left\{ \overbrace{\frac{2 \left\lceil \frac{2\gamma}{m+\ell} \right\rceil + 1}{\left\lceil \frac{2\gamma}{u} \right\rceil + 1}}^{R' \text{ from Theorem 5}}, \overbrace{\frac{\left\lceil \frac{2\gamma}{\ell} \right\rceil + 1}{2 \left\lceil \frac{2\gamma}{u+m} \right\rceil}}^{R'' \text{ from Theorem 6}} \right\} = R^{ls}$$

Lastly, we introduce the modified LPT-SPT (M-LPT-SPT) sequencing rule, which is built from an LPT-SPT schedule. To construct an M-LPT-SPT schedule, we select pods in pairs from alternating ends of the LPT-SPT schedule and schedule the pods in this new order. Specifically, we start with the two leftmost pods in the LPT-SPT schedule, followed by the two rightmost, and continue this pattern—excluding pods already scheduled—until all pods are assigned. We provide a working example for the M-LPT-SPT rule in Section 6. We note that the theoretical performance bounds we provided above for the LPT-SPT rule, given in Theorems 5, 6, and Corollary 1, hold true for M-LPT-SPT rule as well, without requiring any modifications to their respective proofs. The reason for this is the structure of the referenced theorems and corollary. In particular, in the proofs of Theorems 5 and 6, we build specific problem instances to derive our bounds. In these specific instances, we consider two distinct integers as pod processing times: ℓ and m for Theorem 5 and m and u for Theorem 6. This structure ensures that the LPT-SPT and M-LPT-SPT schedules are identical for our specific problem instances, allowing us to use the same derived performance bounds for both LPT-SPT and M-LPT-SPT.

In the e-companion, we devise a novel NRR minimization formulation to find the optimal NRR for the single-picker case, considering multiple consecutive picking intervals (i.e., a horizon). This formulation allows us to perform computational experiments to evaluate the average-case performance bounds of the LPT-SPT rule on practically relevant problem instances. The formulation and the average-case performance bound results are presented in Section EC.6. The average-case performance bounds suggest that the LPT-SPT rule performs much better than its theoretical worst-case bound R^{ls} .

Another NRR minimization formulation similar to the one given in Section EC.6 will be presented in Section 6.2, where we introduce a hypothetical perfect information case to serve as a lower bound in our computational experiments. Both of these two novel formulations are derived from specific theoretical properties of the schedule distance matrices (such as the one given in Example 2), explained next. We first define the concept of the m th diagonal of any upper triangular $r \times r$ matrix A , with an example in Section EC.7.

DEFINITION 5. For an upper triangular $r \times r$ matrix A with the element a_{ij} in its i th row and j th column, we define the m th diagonal ($m \leq r$) as the tuple of elements $\text{dia}(A, m) = (a_{i,i+m-1})_{i=1}^{r-m+1}$.

The following lemma constitutes the main premise of our novel NRR minimization formulations in Sections EC.6 and 6.2. The lemma (with its proof in Section EC.7) establishes that it can be checked if the number of robots required to support the horizon schedule is exactly m (i.e., if $f_{\text{NRR}}(\eta; \pi) = m$) by scanning the m th and $(m+1)$ th diagonals of $E(\eta; \pi)$ to see if they include negative elements.

LEMMA 2. Let $\pi = [\pi^1, \pi^2, \dots, \pi^T]$ be a given horizon schedule for one picker. Furthermore, let $E = E(\eta; \pi) = M(\eta; \pi) - 2\gamma$ be the matrix generated based on distances between pods in the horizon schedule π (cf. Example 2). Then, $f_{\text{NRR}}(\eta; \pi) = m$ if and only if there exists a negative element in $\text{dia}(E, m)$ and there does not exist a negative element in $\text{dia}(E, m+1)$.

4.3 Restricted Problem Forms Solvable in Polynomial Time

We have established that PodSched-OCT(k), when $k \geq 2$, and PodSched-NRR(k), when $k \geq 1$, are NP-hard problems, meaning that the stand-alone pod scheduling problem, under either objective, cannot be solved optimally in polynomial time when there are two or more pickers in the system. We now introduce two restricted forms of PodSched-OCT(k) and PodSched-NRR(k) that can be solved in polynomial time even when $k \geq 2$.

Restricted Form A. This restricted form, associated with a pod processing times set $\eta_d^t = \{\eta_d^t \mid d \in \mathcal{D}(t)\}$, requires that the number of pods to process in an interval t , denoted $|\mathcal{D}(t)| = D(t)$, does not exceed the number of pickers in the system, i.e., $D(t) \leq k$. Under these conditions, solving PodSched-OCT(k) is straightforward: any $D(t)$ pickers from the available k can be randomly selected, each to process one randomly chosen pod while the remaining $k - D(t)$ pickers do not process any pods. Let π_+^t denote any such schedule. In this case, π_+^t is clearly optimal in minimizing OCT, and the optimal OCT value is given by the longest processing time among the pods to process, i.e., $f_{\text{OCT}}(\eta_d^t; \pi_+^t) = \max\{\eta_d^t \mid d \in \mathcal{D}(t)\}$. Similarly, fixing the found optimal OCT, π_+^t is also an optimal solution to PodSched-NRR(k), where the optimal NRR value is given by $f_{\text{NRR}}(\eta_d^t; \pi_+^t) = D(t)$. This follows from the fact that each pod scheduled for processing in interval t is handled concurrently by a different picker, necessitating a separate robot for each pod without any possibility of sharing robots among the pickers.

Restricted Form B. This restricted form, associated with a pod processing times set $\eta_B^t = \{\eta_d^t \mid d \in \mathcal{D}(t)\}$, requires all pods processed in interval t to have the same pod processing time, i.e., $\eta_d^t = \eta^t$ for all $d \in \mathcal{D}(t)$. Any ordering of the

$|\mathcal{D}(t)| = D(t)$ pods is identical in this case, and any schedule π_+^t obtained by assigning the next pod in the sequence to the first available picker will be an optimal solution to PodSched-OCT(k), as this method balances the number of pods assigned to each picker to the extent possible. So, it also follows that the LPT rule is optimal for this case.

In particular, let $\rho \geq 0$ be the integer remainder obtained by dividing $D(t)$ by k .⁸ Then, ρ pickers will process $\lceil D(t)/k \rceil$ pods and the remaining $(k - \rho)$ pickers will process $\lfloor D(t)/k \rfloor$ pods in schedule π_+^t and the optimal OCT will be $f_{\text{OCT}}(\eta_B^t; \pi_+^t) = \eta^t \times \lceil D(t)/k \rceil$.⁹ Fixing the found optimal OCT, we note that π_+^t is also an optimal solution to PodSched-NRR(k), as perturbing the order of pods assigned to a picker would not have any effect on the number of robots required to support the picker's schedule as $\eta_d^t = \eta^t$ in this restricted form. Next, we provide a theorem establishing that the overall NRR can be computed with a closed-form equation in this case, with its proof in Section EC.8.

THEOREM 7. For Restricted Form B, associated with a pod processing time set η_B^t , let $\rho \geq 0$ denote the remainder when the number of pods, $D(t)$, is divided by the number of pickers, k . Additionally, let π_+^t represent any schedule constructed by sequentially assigning each pod to the first available picker under Restricted Form B. The number of robots required to support π_+^t is then given by:

$$f_{\text{NRR}}(\eta_B^t; \pi_+^t) = \rho \times \min \{2 + \lfloor 2\gamma/\eta^t - \epsilon \rfloor, \lceil D(t)/k \rceil\} + (k - \rho) \times \min \{2 + \lfloor 2\gamma/\eta^t - \epsilon \rfloor, \lfloor D(t)/k \rfloor\},$$

where $1 \gg \epsilon > 0$ is a small positive decimal and γ is the one-way robot travel time between the pod storage area and pickers.

Theorem 7 provides a means to determine a lower bound on the optimal NRR value for any PodSched-NRR(k) instance. Given an instance with a pod processing times set η^t and one-way robot travel time γ , a Restricted Form B instance can be constructed by setting all pod processing times to the maximum value in η^t and keeping the same γ value. Then, the NRR value obtained from Theorem 7 with this modified problem instance serves as a lower bound to the optimal NRR of the original instance.

5 Incorporating the Pod Selection Problem

In this section, we investigate how the pod selection problem connects to the pod scheduling problem and consider the study's primary objective of minimizing OCT. Pod selection and scheduling processes follow each other in practice. First, based on a known inventory allocation scheme with a set number of pods and SKU-level stock information for each pod,¹⁰

the system identifies which pods to use within each picking interval and the number of items to pick from each pod to meet the assigned picking demand. Second, the selected pods are scheduled with the pickers.

We provide two primary approaches to implement the pod selection process. First, as a scalable heuristic for large problem instances, the pod selection problem can be formulated and solved using an intuitive objective, such as minimizing the number of pods sent to pickers within an interval (Boysen et al., 2017; Xie et al., 2021). The selected pods can then be scheduled using a rule like the LPT rule, which has been shown to be asymptotically optimal in minimizing OCT with multiple pickers (cf. Section 4.1). We call this first approach *sequential pod selection and scheduling* (SPS). Second, the pod selection and scheduling problems can be solved together within a single formulation to simultaneously find solutions to both in an integrated manner. We label this second approach as *integrated pod selection and scheduling* (IPS).

We note that the OCT of a schedule is determined by the first component of the scheduling problem (pod-to-picker assignment) rather than the second component (pod sequencing). This means that as long as the pod-to-picker assignments remain unchanged, the processing sequence of pods for each picker can be altered without affecting the OCT. We will leverage this concept in Section 6 to minimize NRR while preserving the OCT levels achieved by either approach—SPS or IPS.

5.1 SPS to Minimize OCT

If the pod scheduling problem is to be solved independently as the SPS approach prescribes, the number of items to pick from each pod $d \in \mathcal{D}$ in the interval t , denoted $\phi_d^t \in \mathbb{Z}_0^+$, must be known in advance. This necessitates a separate formulation for the pod selection problem, i.e., the introduction of a stand-alone pod selection problem. Given that the overall objective is to minimize OCT, it is essential to identify a suitable objective for this stand-alone pod selection problem. Following Boysen et al. (2017) and Xie et al. (2021), we formulate the stand-alone pod selection problem to minimize the number of pods sent to pickers within a picking interval. This objective helps minimize OCT during the pod scheduling step because each pod used incurs a setup time β , so reducing the number of pods selected contributes to lowering the OCT. The stand-alone pod selection problem is therefore defined as follows.

DEFINITION 6. Given a picking interval t , the item-picking demands assigned to it $\{(i, m_i^t) \mid i \in \mathcal{I}\}$, and the stock information at the SKU-pod level $\{\Lambda_{id}^t \mid i \in \mathcal{I}, d \in \mathcal{D}\}$, the stand-alone pod selection problem is to determine the number of items of each SKU to be picked from each pod so that all item-picking demands are satisfied and the number of pods sent to pickers during t is minimized.

We provide an integer programming formulation for this problem, involving two sets of variables. First, the integer

variable x_{id} represents the number of items of SKU i to be picked from pod d . Second, the binary variable y_d indicates whether pod d is processed in interval t (1 if processed, 0 otherwise). The following formulation is then solved for a picking interval t .

$$\min \sum_{d \in \mathcal{D}} y_d \quad (4)$$

$$\text{s.t.} \quad \sum_{d \in \mathcal{D}} x_{id} \geq m_i^t \quad \forall i \in \mathcal{I} \quad (5)$$

$$\Lambda_{id}^t y_d - x_{id} \geq 0 \quad \forall d \in \mathcal{D}, \forall i \in \mathcal{I} \quad (6)$$

$$x_{id} \geq 0, \quad x_{id} \in \mathbb{Z} \quad \forall d \in \mathcal{D}, \forall i \in \mathcal{I} \quad (7)$$

$$y_d \in \{0, 1\} \quad \forall d \in \mathcal{D} \quad (8)$$

The objective function (4) minimizes the total number of pods processed during picking interval t . Constraints (5) ensure that all item-picking demands assigned to t are fulfilled. Constraints (6) limit the picked quantity of an SKU from a pod to the SKU's available inventory in that pod. Finally, constraints (7) and (8) define the non-negative integer and binary domains for the variables x_{id} and y_d , respectively. After solving this problem to optimality, the solution can be processed to determine the number of items to pick from each pod $d \in \mathcal{D}$, denoted as $\phi_d^t \in \mathbb{Z}_0^+$. Note that we defined $\mathcal{D}(t) := \{d \mid d \in \mathcal{D}, \phi_d^t > 0\}$ as the pods to be processed in interval t , where $\mathcal{D}(t) \subseteq \mathcal{D}$. The next step is to solve a stand-alone pod scheduling problem for the interval t . In SPS, we solve this stand-alone pod scheduling problem with the LPT rule (cf. Section 4.1).

5.2 IPS to Minimize OCT

In this section, we incorporate the pod selection and pod scheduling problems into a single mixed-integer program formulation. This formulation is designed to be solved at the start of each interval t to minimize the interval's OCT. The inputs to this formulation include the item-picking demands assigned to t , represented by the set $\{(i, m_i^t) \mid i \in \mathcal{I}\}$, which are derived from orders released to the system at the beginning of the interval. Additionally, the inventory allocation scheme, represented by the stock information at the SKU-pod level Λ_{id}^t for all $i \in \mathcal{I}$ and $d \in \mathcal{D}$, is known at the start of interval t .

The decision variables for this formulation include x_{idp} , which denotes the number of SKU $i \in \mathcal{I}$ items that picker $p \in \mathcal{P}$ will pick from pod $d \in \mathcal{D}$, and y_{dp} , a binary variable indicating whether pod $d \in \mathcal{D}$ is processed by picker $p \in \mathcal{P}$ ($y_{dp} = 1$) or not ($y_{dp} = 0$). The OCT for the focal picking interval is denoted by the variable f . The formulation is given as follows.

$$\text{(IPS)} \quad \min \quad f \quad (9)$$

$$\text{s.t.} \quad \sum_{p \in \mathcal{P}} x_{idp} \leq \Lambda_{id}^t \quad \forall i \in \mathcal{I}, \forall d \in \mathcal{D} \quad (10)$$

$$\sum_{d \in \mathcal{D}} \sum_{p \in \mathcal{P}} x_{idp} = m_i^t \quad \forall i \in \mathcal{I} \quad (11)$$

$$\sum_{p \in \mathcal{P}} y_{dp} \leq 1 \quad \forall d \in \mathcal{D} \quad (12)$$

$$\sum_{i \in \mathcal{I}} x_{idp} \leq c y_{dp} \quad \forall d \in \mathcal{D}, \forall p \in \mathcal{P} \quad (13)$$

$$\sum_{i \in \mathcal{I}} \sum_{d \in \mathcal{D}} \alpha x_{idp} + \sum_{d \in \mathcal{D}} \beta y_{dp} \leq f \quad \forall p \in \mathcal{P} \quad (14)$$

$$x_{idp} \geq 0, \quad x_{idp} \in \mathbb{Z} \quad \forall i \in \mathcal{I}, \quad \forall d \in \mathcal{D}, \quad \forall p \in \mathcal{P} \quad (15)$$

$$y_{dp} \in \{0, 1\} \quad \forall d \in \mathcal{D}, \forall p \in \mathcal{P} \quad (16)$$

$$f \geq 0 \quad (17)$$

The objective is to minimize the OCT of the focal interval t , as specified in objective (9). Constraints (10) ensure that the quantities picked for each SKU from each pod do not exceed the available inventory Λ_{id}^t . Constraints (11) guarantee that the total quantity picked for each SKU meets the demand m_i^t for the interval. Constraints (12) tighten the formulation by restricting solutions to those with a single picking run (cf. Lemma 1). Constraints (13) link the x and y variables, stipulating that if $y_{dp} = 1$, picker p can pick from pod d up to the pod's capacity, c . The variable f used in the objective function is given lower bounds by constraints (14), while the domain constraints for the variables are given in (15), (16), and (17).

Both approaches (SPS and IPS) clearly demonstrate that once the pod-to-picker assignment problem is introduced, the structure of pod scheduling with OCT minimization diverges significantly from that of makespan minimization with multiple machines. In our context, the number and processing times of pods to be processed (analogous to job processing times in machine scheduling) are determined by the pod selection solution. In contrast, in makespan minimization, these parameters are exogenous. Additionally, our context introduces the complexity of considering inventory levels in each pod and the specific demand that must be met, further differentiating it from traditional makespan minimization problems.

6 Sequencing of Pods Assigned to a Picker

Once the pod-to-picker assignments are determined using either sequential or integrated pod selection and scheduling approaches, the OCT for the focal interval is established. With the OCT fixed, the next step is to decide the pod sequences for each picker to minimize NRR without altering the pod-to-picker assignments. We present two rules to accomplish this: LPT-SPT and M-LPT-SPT.

6.1 LPT-SPT and M-LPT-SPT Rules

The LPT-SPT sequencing rule offers a simple yet effective method for building pod sequences from given pod-to-picker assignments. For a given interval t , picker p , and the pods assigned to p for t , LPT-SPT arranges the pods by selecting the one with the longest processing time as the first pod to process,

followed by the one with the shortest processing time and then alternates between the next longest and shortest until all pods are arranged. This alternating approach helps balance the pod sequence for each picker, preventing robots from handling too many consecutive pods with short processing times, thereby reducing NRR. Section 4.2 has already established some theoretical properties of the LPT-SPT rule within the context of multiple consecutive picking intervals. Here, let us consider an example of how this rule can be utilized to sequence pods assigned to a picker.

EXAMPLE 3. Assume that for an interval t in an RMF system with $k = 3$ pickers, we apply either SPS or IPS to obtain the pod-to-picker assignments as shown in Table 4. This table also presents the number of items to be picked from each pod d , denoted as ϕ_d^t , as determined by either approach. Then, assuming a unit picking time of $\alpha = 1$ minute per item and a pod setup time of $\beta = 3$ minutes, we can calculate the processing time for each pod d , denoted as η_d^t , which is provided in the last row of Table 4.

From this table, the OCT of this picking interval can be calculated as $\max\{80, 77, 79\} = 80$. One can sequence the pods assigned to each picker with the LPT-SPT rule to obtain the interval schedule π^t shown in Figure 4. If the travel time between pickers and storage area is $\gamma = 14$ minutes, then NRR is 15 for this schedule.

M-LPT-SPT is a modified version of LPT-SPT. To create an M-LPT-SPT schedule, we begin with the LPT-SPT schedule and then start selecting pairs of pods from the left and right sides of the LPT-SPT schedule and build the new schedule in this order. In other words, starting with a picker, we take two pods from the leftmost side of the LPT-SPT schedule and assign them as the first and second pods in the new M-LPT-SPT schedule. Next, we take the two rightmost pods from the LPT-SPT schedule and assign them as the third and fourth pods in the new schedule. Continuing this pattern, we select the next two leftmost pods, excluding those already scheduled, and place them as the fifth and sixth pods in the new schedule. This alternating selection and scheduling process repeats until all pods are scheduled, finally leading to the complete M-LPT-SPT schedule. For example, for $p = 1$ in the LPT-SPT schedule shown in Figure 4, we first select the pair $\{d = 3, d = 6\}$ from the left, then $\{d = 8, d = 2\}$ from the right, followed by $\{d = 7, d = 4\}$ from the left, and finally $\{d = 5, d = 1\}$ from the right, constructing the M-LPT-SPT schedule for picker $p = 1$ in this way. When we apply the same process for all three pickers in this example, we obtain the M-LPT-SPT schedule given in Figure 5. If the robot travel time between pickers and storage area is $\gamma = 14$ minutes, the NRR is still 15 for this M-LPT-SPT schedule.

Table 4. Pod-to-picker assignments for Example 3.

Picker	$p = 1$								$p = 2$								$p = 3$							
d	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
ϕ_d^t	4	6	12	4	8	4	10	8	4	6	9	6	6	12	5	5	7	5	12	3	4	6	6	12
η_d^t	7	9	15	7	11	7	13	11	7	9	12	9	9	15	8	8	10	8	15	6	7	9	9	15

$p = 1$	$\eta_3^t = 15$	$\eta_6^t = 7$	$\eta_7^t = 13$	$\eta_4^t = 7$	$\eta_5^t = 11$	$\eta_1^t = 7$	$\eta_8^t = 11$	$\eta_2^t = 9$
$p = 2$	$\eta_{14}^t = 15$	$\eta_9^t = 7$	$\eta_{11}^t = 12$	$\eta_{16}^t = 8$	$\eta_{12}^t = 9$	$\eta_{15}^t = 8$	$\eta_{10}^t = 9$	$\eta_{13}^t = 9$
$p = 3$	$\eta_{19}^t = 15$	$\eta_{20}^t = 6$	$\eta_{24}^t = 15$	$\eta_{21}^t = 7$	$\eta_{17}^t = 10$	$\eta_{18}^t = 8$	$\eta_{22}^t = 9$	$\eta_{23}^t = 9$

Figure 4. The longest processing time-shortest processing time (LPT-SPT) interval schedule π^t built for Example 3.

$p = 1$	$\eta_3^t = 15$	$\eta_6^t = 7$	$\eta_8^t = 11$	$\eta_2^t = 9$	$\eta_7^t = 13$	$\eta_4^t = 7$	$\eta_5^t = 11$	$\eta_1^t = 7$
$p = 2$	$\eta_{14}^t = 15$	$\eta_9^t = 7$	$\eta_{10}^t = 9$	$\eta_{13}^t = 9$	$\eta_{11}^t = 12$	$\eta_{16}^t = 8$	$\eta_{12}^t = 9$	$\eta_{15}^t = 8$
$p = 3$	$\eta_{19}^t = 15$	$\eta_{20}^t = 6$	$\eta_{22}^t = 9$	$\eta_{23}^t = 9$	$\eta_{24}^t = 15$	$\eta_{21}^t = 7$	$\eta_{17}^t = 10$	$\eta_{18}^t = 8$

Figure 5. The modified longest processing time-shortest processing time (M-LPT-SPT) interval schedule π^t built for Example 3.

6.2 Perfect Information Benchmark for Sequencing Rules

The sequencing rules introduced here are applied within an interval, meaning that when LPT-SPT or M-LPT-SPT is used in interval t , the system lacks information about demands in future intervals ($t + 1$, $t + 2$, etc.). We now introduce a *perfect information* case, where the system knows the demands for all intervals in $\mathcal{T}$ from the start of the first interval $t = 1$. We then present a novel integer programming formulation for this case. Although unattainable in RMF systems, results from this case will serve as lower bounds in our computational experiments, providing a benchmark for the two sequencing rules.

Here, we will minimize the number of required robots to support a single picker's horizon schedule. We do this minimization for all pickers $p \in \mathcal{P}$ to minimize NRR in a planning horizon for the whole system. Decomposing the problem into individual pickers is required for the tractability of this otherwise very complex and intractable problem. We are assuming here that the pod processing times for all intervals, i.e., $\eta = (\eta^1, \eta^2, \dots, \eta^T)$, are known at the start of the planning horizon $\mathcal{T}$. Lemma 2 is the main premise of the perfect information integer program we will present here. This lemma established that it can be checked if NRR is exactly m by scanning the m th and $(m + 1)$ th diagonals of a schedule's distance matrix $E(\eta; \bar{\pi})$ to see if they include negative elements.

Let D_h denote the total number of non-zero elements across sets $\eta^1, \eta^2, \dots, \eta^T$. In other words, D_h is the number of pods to process in horizon $\mathcal{T}$, where a pod is treated as different pods in different intervals in this notation. We define the set $\mathcal{D}_h := \{1, 2, \dots, D_h\}$. Let $\mathcal{D}_h(t) \subset \mathcal{D}_h$ be the set of pods to be processed in interval t . In the formulation that follows, we

distinguish between a *pod* and a *sequence position* (henceforth, *position*). A pod is an element d in the set $\mathcal{D}_h$, i.e., $d \in \mathcal{D}_h = \{1, 2, \dots, D_h\}$. Similarly, a position can take the values of $j \in \mathcal{D}_h = \{1, 2, \dots, D_h\}$. To this end, our formulation assigns D_h pods to D_h positions to minimize NRR of a horizon while ensuring that a pod to be processed in interval t is assigned to schedule positions in t , denoted as $\Theta_h(t)$.

The formulation includes three types of decision variables: x -, y -, and z -types. First, x_j is a continuous variable representing the processing time of the pod assigned to the j th position. Second, y_{dj} is a binary variable, equaling 1 if pod d is assigned to the j th position and 0 otherwise. Lastly, for a schedule $\bar{\pi}$, z_m is a binary variable taking a value of 1 if the m th diagonal of $E(\eta; \bar{\pi})$ includes a negative element, and 0 otherwise, where $m \in \{1, 2, \dots, D_h\}$. We present the perfect information case formulation as follows.

$$z^* = \min \sum_{m \in \mathcal{D}_h} z_m \quad (18)$$

subject to:

$$2\gamma z_m + \sum_{v=j}^{j+m-3} x_v \geq 2\gamma \quad j = 2, 3, \dots, (D_h - m + 2); \quad m = 3, 4, \dots, D_h \quad (19)$$

$$z_m \geq z_{m+1} \quad m = 1, 2, \dots, D_h - 1 \quad (20)$$

$$\sum_{j \in \Theta_h(t)} y_{dj} = 1 \quad \forall d \in \mathcal{D}_h(t); \quad \forall t \in \mathcal{T} \quad (21)$$

$$\sum_{d \in \mathcal{D}_h(t)} y_{dj} = 1 \quad \forall j \in \Theta_h(t); \quad \forall t \in \mathcal{T} \quad (22)$$

$$x_j - \sum_{d \in \mathcal{D}_h} \eta_d^t y_{dj} = 0 \quad \forall j \in \mathcal{D}_h \quad (23)$$

$$z_1 = 1; \quad z_2 = 1 \quad (24)$$

$$x_j \geq 0 \quad \forall j \in \mathcal{D}_h \quad (25)$$

$$y_{dj} \in \{0, 1\} \quad \forall d \in \mathcal{D}_h; \quad \forall j \in \mathcal{D}_h \quad (26)$$

$$z_m \in \{0, 1\} \quad \forall m \in \mathcal{D}_h \quad (27)$$

In the above formulation, the objective (18) is to minimize the number of robots required to support a picker's horizon schedule. To this end, constraints (19) and (20) together ensure that either all elements of the m th diagonal in $E(\eta; \tilde{\pi})$ are non-negative, or NRR is at least m , as follows from Lemma 2. Constraints (21) ensure that a pod to be processed in an interval t is assigned to a position in t . Constraints (22) guarantee that each position in interval t is assigned exactly one pod to be processed in t . Constraints (23) connect x - and y -type variables. These constraints use the parameters η_d^t for all $d \in \mathcal{D}(t)$ and $t \in \mathcal{T}$ to incorporate pod processing times into the model. Constraints (24) state that the first two diagonals always contain negative elements, regardless of the solution—e.g., Example 2 and the matrix given in (3) therein. Lastly, constraints (25), (26), and (27) set the variable domains.

7 Computational Study

This section presents our computational study using order data from JD.com (Shen et al., 2020). Our analysis focuses on an individual fulfillment center that processed 21,834 customer orders in a month, with each customer order containing approximately 1.15 unique SKUs and 1.35 items on average. For our study, we consider a single RMF system operating within this fulfillment center.

To increase the order volume and create a larger, more complex RMF system, we expand the data by resampling. First, we decompose customer orders into SKUs they contain, resulting in a total of 25,096 decomposed orders. This transformation ensures that the resampling process more accurately preserves the underlying SKU demand distributions, as opposed to resampling multiple SKUs as inseparable blocks.¹¹ Second, we further increase the number of orders by a factor of three by random sampling with replacement. Expanding the data in this manner helps highlight performance differences in the pod selection and scheduling strategies through our computational study.¹²

We assume the RMF system releases batches of 600 orders at the start of an interval to be picked within the interval, yielding $3 \times 25,096/600 \approx 125$ picking intervals in the expanded dataset. To facilitate our experiments, we need to assume specific inventory levels at the start of each interval. In practice, an order is released only if sufficient inventory is available at the time of release (the second assumption in Section 3). Since the data do not include inventory details, we ensure the RMF system in our study has sufficient inventory for each SKU. In particular, for each SKU, we set the initial inventory at 2.5

times the highest demand observed across 125 intervals. The impact of stockouts will be examined in Section 7.4 as part of our sensitivity and robustness analysis.

As inventory allocation and replenishment fall outside the scope of our research, we present our computational study results under two inventory allocation policies as controls: (i) Random and (ii) similarity-based. The random policy reflects real-world practices where inventory is distributed in the order of arrival at the fulfillment center without optimization. Amazon uses this policy in its RMF systems (Allgor et al., 2023). In contrast, the similarity-based policy, which is favored in the RMF literature, assigns frequently picked SKUs to the same pod to reduce the number of pods needed in an interval, potentially improving picking efficiency. Sections EC.9 and EC.10 present the implementation details for the random and similarity-based inventory allocation policies, respectively. Since the similarity-based inventory policy requires two interval sets (training and test), in what follows, we divide the 125 picking intervals into a training set of 50 intervals and a test set of 75 intervals.

Regardless of the type of inventory policy applied, we start all intervals with the selected inventory scheme and full inventory to establish control. This approach allows us to control for variability in the inventory schemes, and ensures that any observed differences in performance are due to the effects of SPS/IPS and LPT-SPT/M-LPT-SPT rather than inconsistencies in initial inventory distribution. In the base case analyses, we assume a pod capacity of $c = 100$ items, a unit item picking time of $\alpha = 1$ minute, and a pod setup time of $\beta = 3$ minutes. In Section 7.4, we examine the impact of varying these parameters.

In what follows, we consider (i) SPS and IPS with respect to their performance in minimizing OCT (Section 7.1), and the sequencing rules, LPT-SPT and M-LPT-SPT, with respect to their performance in minimizing NRR (Section 7.2). This computational study flow is depicted in Figure 6 for ease of reference. Furthermore, in Section 7.3, we provide managerial guidelines on determining an appropriate number of pickers for RMF systems based on a target OCT value. Lastly, in Section 7.4, we present our robustness and sensitivity analysis with six extensions.

7.1 Sequential vs. IPS to Minimize OCT

We apply SPS and IPS to each interval in the test set, which consists of 75 intervals. These 75 intervals are then grouped into 15 planning horizons, with each horizon containing five intervals, to evaluate the performance of the proposed approaches when multiple intervals are considered consecutively. Intervals within the same planning horizon are concatenated using the junction method (Section EC.11).

Table 5 presents the mean OCT and computation times (CPU) per horizon for different picker counts ($k = 7$ to $k = 12$) using SPS and IPS under both random and similarity-based inventory allocation policies. To briefly recap the two

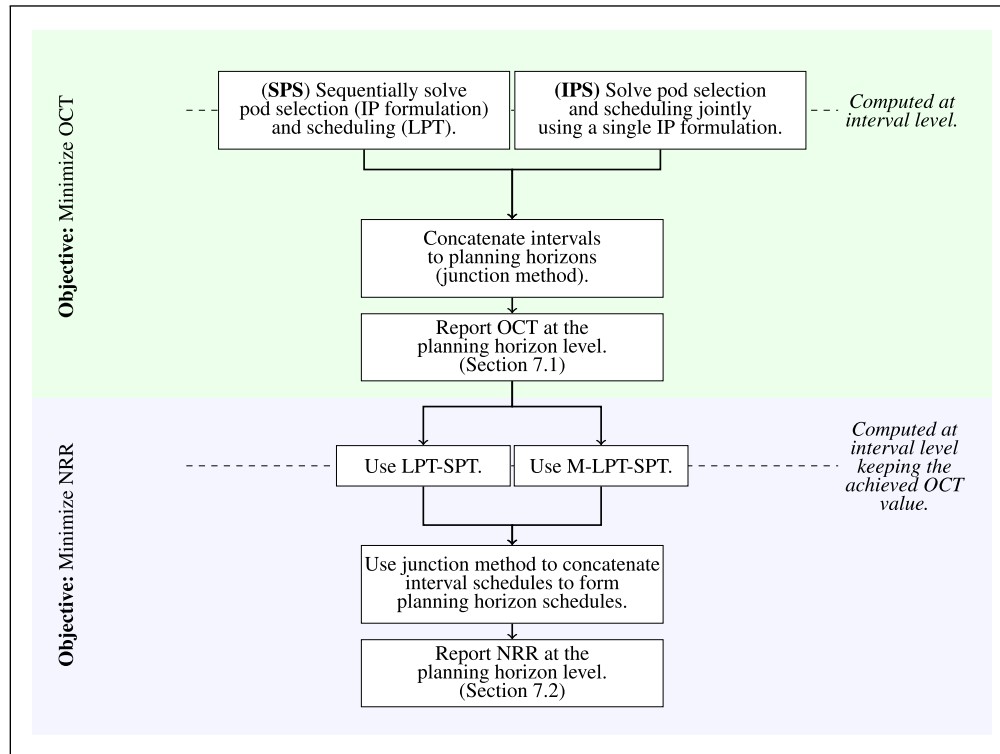


Figure 6. The flow of the computational study section.

Table 5. Mean OCT and CPU per horizon using SPS and IPS.

	Random Inventory						Similarity-Based Inventory					
	OCT			CPU			OCT			CPU		
	SPS	IPS	Diff (%)	SPS	IPS	Diff (%)	SPS	IPS	Diff (%)	SPS	IPS	Diff (%)
$k = 7$	826.87	825.60	0.15	7.28	26.77	267.66	812.60	811.73	0.11	6.83	25.02	266.53
$k = 8$	724.53	722.87	0.23	7.29	30.92	323.86	711.93	710.60	0.19	6.91	28.64	314.44
$k = 9$	644.20	643.07	0.18	7.53	36.31	382.00	633.07	631.73	0.21	6.93	35.55	413.06
$k = 10$	580.20	579.00	0.21	7.29	38.72	431.04	570.00	569.07	0.16	6.90	40.54	487.94
$k = 11$	527.47	526.33	0.22	7.27	41.36	468.99	518.67	517.47	0.23	6.93	48.40	598.77
$k = 12$	483.40	482.67	0.15	7.30	47.85	555.90	475.93	474.73	0.25	6.85	48.12	602.03

Note. k denotes the number of pickers. OCT is in minutes; CPU is in seconds.

Diff (%) is calculated as (Larger Value – Smaller Value) / (Smaller Value) $\times$ 100. OCT = overall completion times; SPS = sequential pod selection and scheduling; IPS = integrated pod selection and scheduling; CPU = computation times.

approaches, SPS treats the pod selection and scheduling problems separately. It first solves a pod selection problem to minimize the number of pods sent to the pickers, followed by using the LPT rule to schedule the selected pods with the available pickers. On the other hand, IPS integrates the pod selection and scheduling problems within the same formulation to minimize the OCT in an interval. We note that while IPS is optimal within an interval, its performance is only asymptotically optimal when the junction method is applied over multiple intervals.

When we compare SPS to IPS in Table 5, we observe that IPS consistently returns a slightly smaller OCT, as expected,

since it addresses the two problems in an integrated manner. However, the differences are minimal across all picker counts and both inventory policies (random and similarity-based). Under the random inventory policy, the percentage difference between the two approaches ranges from 0.15% to 0.23%, remaining less than half a percent. With the similarity-based inventory policy, the percentage difference is nearly the same, ranging from 0.11% to 0.25%, also staying well below half a percent.

When comparing the computation times (CPU), we see that SPS maintains a relatively low and constant computation time

across different picker counts (around 7–8 seconds per horizon). In contrast, the computation time of IPS increases as the number of pickers increases. This suggests that while IPS may offer slightly better OCT performance, the difference is marginal and comes at a significant computational cost. Given the importance of speed in real-time decision-making (since solutions must be generated at the start of each interval) in the RMF context, SPS is preferable due to its much faster computation times, with only up to a 0.25% compromise in OCT performance. In particular, as the number of pickers increases, IPS becomes significantly slower than SPS, taking up to seven times longer to solve the pod selection and scheduling problems. Additionally, while there are 2,623 unique SKUs in our selected mid-size facility, we are aware of larger fulfillment centers that manage tens of thousands of SKUs. Note that IPS would be significantly harder to solve to optimality for these larger systems as the number of variables grows (cf. Section 5.2). Conversely, SPS would still return comparable solutions in near-constant computation times.

7.2 Performance of Sequencing Rules in Minimizing NRR

After achieving our primary goal of minimizing OCT with either SPS or IPS, we proceed to sequence the pods for each picker, maintaining the previously determined pod-to-picker assignments to keep the achieved OCT level. This approach allows us to decompose the scheduling problem (with NRR minimization objective) to individual pickers, enabling us to consider only the pod-processing sequence for each picker. Retaining the pod-to-picker assignments and addressing the NRR minimization problem at the individual picker level is necessary for a practically relevant and applicable approach, as solving NRR minimization for multiple pickers simultaneously would otherwise constitute a highly complex combinatorial problem with the solution space growing exponentially with each added picker.¹³ Hence, in this sequencing phase, we apply one of two sequencing rules, LPT-SPT or M-LPT-SPT, for each picker. This also means our theoretical results of worst-case bounds given in Section 4.2 are relevant here. For brevity, we report only the results where SPS is used to minimize OCT, as it achieves similar OCT performance to IPS but with considerably faster computation times.

To benchmark the performance of these sequencing rules, we compare the results against a lower bound established by a perfect information case. In this case, we assume that the demands of all five intervals in a horizon are known at the start of the first interval, allowing us to apply the formulation presented in Section 6.2. This perfect information scenario provides lower bounds that are unachievable in real-life RMF applications, as the demands of future intervals are unknown when making decisions at the start of the current interval. Hence, the lower bound serves as an idealized comparison point for the results obtained using LPT-SPT and M-LPT-SPT.

Solving the pod selection and scheduling problems with the SPS approach already provides pod-to-picker assignment and pod sequences for each picker. However, we are also interested in assessing how much LPT-SPT and M-LPT-SPT decrease the NRR values compared to the original sequences generated by SPS. By analyzing the differences, we can quantify the extent to which sequencing strategies reduce NRR. To facilitate this comparison, we include the NRR values from the original SPS-generated sequences in our result tables under the label *No re-seq.*

Since robot travel time can significantly impact the NRR metric, we examine scenarios with varying one-way travel times: $\gamma = 10, 12, 14$, and 16 minutes. These values were selected to reflect reasonable pod processing time to travel time ratios, ensuring that the selected travel times represent practical operational scenarios. The results are presented in Tables 6 to 9, which report the mean of the 15 planning horizon NRR values.

7.2.1 NRR Impact of Sequencing. We highlight the NRR minimization advantage achieved by the LPT-SPT and M-LPT-SPT rules compared to the case where no sequencing rules are applied on the original SPS schedule (see *No re-seq.* columns in Tables 6 to 9). The best-performing sequencing rule (LPT-SPT or M-LPT-SPT, depending on the considered experiment setting) can reduce NRR by up to 16.7%, 28.4%, 25.2%, and 23.5% for $\gamma = 10$, $\gamma = 12$, $\gamma = 14$, and $\gamma = 16$ minutes, respectively, compared to not applying any sequencing rules to the schedules obtained by SPS. The mean and minimum reductions across all instances with all γ values are 20.1% and 12.7%, respectively. These magnitudes demonstrate the significant impact that effective sequencing can have on the NRR metric, even if the pod-to-picker assignments are kept fixed to maintain the lowest OCT level.

7.2.2 Comparison of LPT-SPT and M-LPT-SPT. Although Tables 6 to 9 show that sequencing rules significantly reduce the number of required robots compared to no re-sequencing, the preferred rule under different conditions is not immediately clear from the presented results. For example, when the one-way robot travel time is $\gamma = 10$ minutes, Table 6 indicates that LPT-SPT and M-LPT-SPT perform equally well in minimizing NRR, regardless of the number of pickers or inventory allocation policy. Table 7 shows that M-LPT-SPT becomes more favorable than LPT-SPT when $\gamma = 12$ minutes. However, Tables 8 and 9 suggest that this advantage does not consistently extend to $\gamma = 14$ and $\gamma = 16$ minutes, where LPT-SPT sometimes outperforms M-LPT-SPT. We observe this is due to the highly left-skewed distribution of pod processing times after solving the pod selection problem. To examine the case of more balanced pod processing times, we conduct additional experiments in Section EC.12, where we consider more balanced distributions and vary the proportion of pods with small and large processing times. The results of these extended experiments, presented in Tables EC.6 to EC.10,

Table 6. Mean NRR of sequencing rules vs. perfect information for $\gamma = 10$.

		Random inventory							Similarity-based inventory						
		Perfect Info.	LPT-SPT		M-LPT-SPT		No re-seq.		Perfect Info.	LPT-SPT		M-LPT-SPT		No re-seq.	
		NRR	NRR	Gap	NRR	Gap	NRR	Gap	NRR	NRR	Gap	NRR	Gap	NRR	Gap
SPS	$k = 7$	28.73	35.00	6.27	35.00	6.27	42.00	13.27	28.33	35.00	6.67	35.00	6.67	42.00	13.67
	$k = 8$	33.07	40.00	6.93	40.00	6.93	48.00	14.93	32.53	40.00	7.47	40.00	7.47	48.00	15.47
	$k = 9$	37.47	45.00	7.53	45.00	7.53	54.00	16.53	36.80	45.00	8.20	45.00	8.20	54.00	17.20
	$k = 10$	41.27	50.00	8.73	50.00	8.73	60.00	18.73	41.20	50.00	8.80	50.00	8.80	60.00	18.80
	$k = 11$	46.13	55.00	8.87	55.00	8.87	66.00	19.87	45.40	55.00	9.60	55.00	9.60	66.00	20.60
	$k = 12$	50.20	60.00	9.80	60.00	9.80	72.00	21.80	49.47	60.00	10.53	60.00	10.53	72.00	22.53

Note. k is the number of pickers; *Gap* refers to the gap between a method and the perfect information case in number of robots. LPT-SPT = longest processing time-shortest processing time; M-LPT-SPT = modified longest processing time-shortest processing time; NRR = number of required robots; SPS = sequential pod selection and scheduling.

Table 7. Mean NRR of sequencing rules vs. perfect information for $\gamma = 12$.

		Random inventory							Similarity-based inventory						
		Perfect Info.	LPT-SPT		M-LPT-SPT		No re-seq.		Perfect Info.	LPT-SPT		M-LPT-SPT		No re-seq.	
		NRR	NRR	Gap	NRR	Gap	NRR	Gap	NRR	NRR	Gap	NRR	Gap	NRR	Gap
SPS	$k = 7$	35.00	35.53	0.53	35.27	0.27	49.00	14.00	35.00	35.27	0.27	35.07	0.07	49.00	14.00
	$k = 8$	40.00	40.80	0.80	40.40	0.40	56.00	16.00	40.00	40.67	0.67	40.27	0.27	56.00	16.00
	$k = 9$	45.00	46.47	1.47	45.87	0.87	63.00	18.00	45.00	45.87	0.87	45.47	0.47	62.93	17.93
	$k = 10$	50.00	52.27	2.27	50.73	0.73	69.93	19.93	50.00	51.60	1.60	50.87	0.87	70.00	20.00
	$k = 11$	55.07	58.27	3.20	56.80	1.73	76.47	21.40	55.00	57.27	2.27	56.33	1.33	76.20	21.20
	$k = 12$	60.13	63.80	3.67	61.87	1.74	81.93	21.80	60.00	63.27	3.27	61.80	1.80	80.53	20.53

Note. LPT-SPT = longest processing time-shortest processing time; M-LPT-SPT = modified longest processing time-shortest processing time; NRR = number of required robots; SPS = sequential pod selection and scheduling.

Table 8. Mean NRR of sequencing rules vs. perfect information for $\gamma = 14$.

		Random inventory							Similarity-based inventory						
		Perfect Info.	LPT-SPT		M-LPT-SPT		No re-seq.		Perfect Info.	LPT-SPT		M-LPT-SPT		No re-seq.	
		NRR	NRR	Gap	NRR	Gap	NRR	Gap	NRR	NRR	Gap	NRR	Gap	NRR	Gap
SPS	$k = 7$	36.53	42.07	5.54	42.13	5.60	56.00	19.47	35.60	42.00	6.40	41.87	6.27	56.00	20.40
	$k = 8$	41.73	48.13	6.40	48.27	6.54	63.80	22.07	40.87	48.07	7.20	47.93	7.06	63.67	22.80
	$k = 9$	47.20	54.27	7.07	54.60	7.40	70.93	23.73	45.87	54.13	8.26	54.07	8.20	70.00	24.13
	$k = 10$	52.13	60.20	8.07	60.47	8.34	76.13	24.00	51.47	60.27	8.80	60.20	8.73	73.47	22.00
	$k = 11$	57.93	67.07	9.14	67.53	9.60	80.40	22.47	56.53	66.73	10.20	66.93	10.40	78.73	22.20
	$k = 12$	62.87	73.13	10.26	73.40	10.53	85.27	22.40	61.40	73.20	11.80	73.07	11.67	84.53	23.13

Note. LPT-SPT = longest processing time-shortest processing time; M-LPT-SPT = modified longest processing time-shortest processing time; NRR = number of required robots; SPS = sequential pod selection and scheduling.

indicate that with a more balanced pod processing time distribution, LPT-SPT outperforms M-LPT-SPT when a moderate proportion of pod processing times are short relative to overall pod processing times. Conversely, when a high proportion of pod processing times are short, M-LPT-SPT performs better in minimizing NRR.

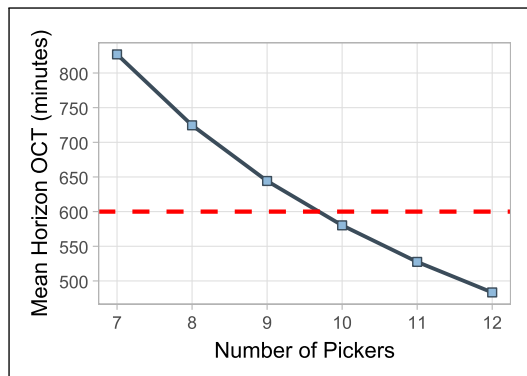
From a managerial perspective, our findings indicate that when pod processing times are highly left-skewed (i.e., when almost all pods have *very short* processing times), the performance gap between LPT-SPT and M-LPT-SPT may be

negligible, suggesting that either rule can be applied without significant trade-offs. However, in balanced distributions (i.e., when the distribution is not highly left-skewed), M-LPT-SPT is preferable when shorter processing pods dominate, while LPT-SPT is the better choice when longer processing pods are more common. Therefore, managers should consider their pod processing time distributions when selecting a sequencing rule.

Table 9. Mean NRR of sequencing rules vs. perfect information for $\gamma = 16$.

		Random inventory								Similarity-based inventory							
		Perfect Info.		LPT-SPT		M-LPT-SPT		No re-seq.		Perfect Info.		LPT-SPT		M-LPT-SPT		No re-seq.	
		NRR		NRR	Gap	NRR	Gap	NRR	Gap	NRR		NRR	Gap	NRR	Gap	NRR	Gap
SPS	$k = 7$	42.00		49.00	7.00	48.47	6.47	61.53	19.53	42.00		47.07	5.07	47.53	5.53	61.53	19.53
	$k = 8$	48.00		55.67	7.67	55.53	7.53	68.20	20.20	48.00		54.13	6.13	54.20	6.20	66.73	18.73
	$k = 9$	54.07		62.73	8.66	62.47	8.40	73.60	19.53	54.00		61.27	7.27	61.13	7.13	72.53	18.53
	$k = 10$	60.07		69.80	9.73	69.60	9.53	80.47	20.40	60.00		68.40	8.40	68.27	8.27	80.00	20.00
	$k = 11$	66.33		77.07	10.74	76.53	10.20	88.07	21.74	66.07		75.07	9.00	74.67	8.60	88.00	21.93
	$k = 12$	72.00		83.80	11.80	84.20	12.20	96.00	24.00	72.00		82.20	10.20	82.53	10.53	96.00	24.00

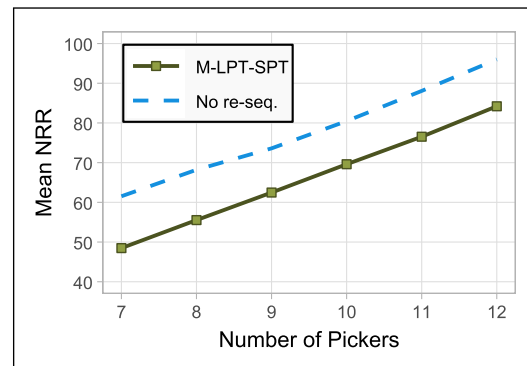
Note. LPT-SPT = longest processing time-shortest processing time; M-LPT-SPT = modified longest processing time-shortest processing time; NRR = number of required robots; SPS = sequential pod selection and scheduling.

**Figure 7.** Mean horizon overall completion time (OCT) for different numbers of pickers.

7.3 Deciding the Number of Pickers for a Target Mean OCT Value

In practice, our computational study design can guide decisions about the number of pickers needed in any RMF system to achieve a target mean horizon OCT value. For the system under consideration, OCT values from our computational study are presented in Figure 7 for different numbers of pickers. For instance, if system management aims for a mean horizon OCT of 600 minutes or less, the figure shows that at least ten pickers are required. While increasing the number of pickers reduces OCT, it also increases the number of robots required. For example, Figure 8 illustrates the mean number of robots required for different picker counts based on (i) the M-LPT-SPT rule (in solid line) or (ii) no re-sequencing case (dashed line), both under the SPS approach, random inventory allocation policy, and a one-way robot travel time of $\gamma = 16$ minutes.

Our proposed sequencing rules (LPT-SPT and M-LPT-SPT) can help RMF system managers strike a balance between OCT and NRR. The target OCT level of 600 minutes required ten pickers in the system. Then, Figure 8 illustrates that ten pickers require around 80 robots, on average, if no sequencing rule is utilized. However, if M-LPT-SPT is used to resequence the pods within each picker's schedule, the mean NRR

**Figure 8.** Mean horizon number of required robots (NRR) for different numbers of pickers.

decreases to approximately 70 robots. Mean NRR constitutes a fundamentally operational metric because it measures the number of robots deployed from a given fleet, *on average*. RMF system managers can also use our computational study structure to determine the fleet size for a particular system by considering a near-worst-case statistic, such as the 95th percentile (Figure 9). This would allow managers to maintain a safety buffer in terms of required robots for most of the planning horizons, while ensuring that the fleet size will be sufficient for 95% of the horizons. This consideration can help managers in two key decision contexts:

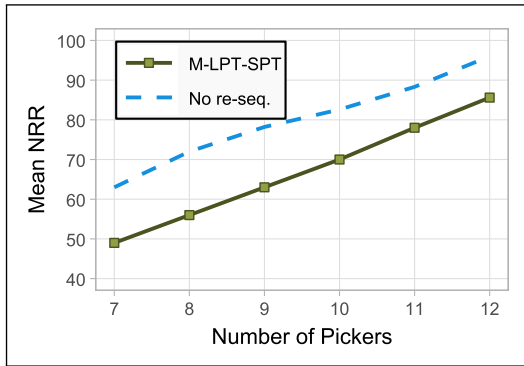
- (i) When an RMF system needs to be operationalized for the first time, some typical demand data is available (or can be estimated), and the management wishes to decide the number of robots to purchase and
- (ii) when there are multiple RMF systems in operation in a single facility, and the robots available in the facility needs to be assigned to individual systems, with little to no robot movement between different RMF systems, as is often the case in practice.

Lastly, we note that although Figures 8 and 9 are given for the SPS approach, random inventory allocation policy, and

Table 10. Computational extensions for robustness and sensitivity Analysis.

#	Base case	Extension	OCT	NRR	Details in §
1	$\alpha = 1, \beta = 3$	Different (α, β) cases	$\uparrow$ as $\alpha \uparrow$ or $\beta \uparrow$	$\downarrow$ as $\alpha \uparrow$ or $\beta \uparrow$	EC.13.1
2	$c = 100$	Different c cases	$\downarrow$ as $c \uparrow$	$\downarrow$ as $c \uparrow$	EC.13.2
3	Deterministic γ	Stochastic γ	N/A	$\uparrow$ as variance $\uparrow$	EC.13.3
4	No worker fatigue	Worker fatigue considered	$\uparrow$ as $\kappa \uparrow$	$\downarrow$ as $\kappa \uparrow$	EC.13.4
5	Perfect pick accuracy	Pick errors possible	$\uparrow$ as error $\uparrow$	$\downarrow$ as error $\uparrow$	EC.13.5
6	Sufficient inventory	Stockout possible	$\leftrightarrow$ as stockouts $\uparrow$	$\leftrightarrow$ as stockouts $\uparrow$	EC.13.6

Note. κ = fatigue coefficient; $\uparrow$ = increases; $\downarrow$ = decreases; $\leftrightarrow$ = no clear trend; N/A = not applicable; OCT = overall completion time; NRR = number of required robots; EC = e-companion.

**Figure 9.** 95th percentile horizon number of required robot (NRR) for different numbers of pickers.

$\gamma = 16$, the results hold qualitatively across all experimental settings in our study.

7.4 Robustness of Results to Varying System Conditions and Relaxed Assumptions

Next, we conduct a robustness and sensitivity analysis to examine how the computational results are impacted by different system conditions and relaxed assumptions. To this end, we define six computational extensions, as summarized in Table 10. We find that the main results of our research consistently hold across all six extensions. In particular, SPS always provides OCT solutions within half a percent of IPS across all extensions while significantly reducing computation times. Furthermore, the best sequencing rule (LPT-SPT or M-LPT-SPT, depending on the case) always reduces mean NRR compared to no-resequencing, with the average reduction ranging from 17.3% to 23.5% in the considered extensions.

We summarize key insights from the computational extensions, with detailed results provided in Section EC.13. First, longer pod processing times reduce the mean NRR values (e.g., in extensions 1, 2, 4, and 5) by spreading out the frequency at which pickers require a new pod at their stations. With more time spent on each pod, robots are under less pressure to deliver the next one quickly, reducing the need for many robots moving pods at the same time. As a result, fewer robots are required to sustain the flow of pods, leading to a lower NRR. Second, relaxing the deterministic robot travel

time (γ) assumption, we see that increases in travel time variance lead to higher NRR (extension 3). Third, OCT increases with longer unit item picking and pod setup times (α and β , respectively), higher fatigue coefficients, and more frequent pick errors (extensions 1, 4, and 5, respectively). However, we observe that OCT decreases with larger pod capacities (c) as larger pods enable a denser inventory allocation, which reduces the number of pods used in a picking interval and, consequently, the number of setup times (β) realized in each interval. This explains why RMF systems favor smaller items that can be densely stored. Fourth, OCT and NRR metrics are not impacted by constant stockout rates (extension 6) when the unfulfilled demand rolls over to the next picking interval. However, we note that frequent stockouts can impact other metrics, such as lead time—i.e., the time between receiving a customer order and the shipment of all items in the order.

8 Conclusion

RMF systems constitute the latest development in fulfillment center automation practices and are becoming more widely adopted, especially in the e-commerce sector. The operational performance of a deployed RMF system plays an important role, as it directly impacts the time and cost savings achieved by the system. To analyze RMF system performance, this research focuses on two key problems: pod selection and pod scheduling. As system metrics, we focus on overall completion time (OCT, i.e., the time point at which all orders assigned to the system for a planning horizon are fulfilled) and the number of required robots (NRR; i.e., to support a given schedule without any unnecessary picker idle times).

We initially focus on the pod scheduling problem independent of the pod selection problem, considering both OCT and NRR metrics, in order. Next, we expand the study to address both pod selection and scheduling problems for multiple pickers, also focusing on our two key metrics in the same order. To minimize OCT, we introduce and investigate two approaches:

- (i) Sequential pod selection and scheduling and
- (ii) Integrated pod selection and scheduling.

The sequential approach solves the pod selection problem first and the pod scheduling problem second. In solving the pod scheduling problem, this approach applies the solutions derived from our theoretical analysis of the stand-alone pod scheduling problem. In contrast, the integrated approach addresses the two problems simultaneously in the same formulation to minimize OCT. The sequential approach offers significantly faster computational times compared to its integrated counterpart while delivering comparable solution qualities. Finally, we re-sequence the schedules produced by these two approaches to reduce NRR while maintaining pod-to-picker assignments to keep the achieved OCT.

Our theoretical contributions to the RMF literature are three-fold. First, to our knowledge, this is the first study to address both pod selection and pod scheduling in RMF using either the OCT or NRR metrics. Notably, the NRR minimization objective introduces a novel optimization problem structure, contributing to scheduling theory even outside the RMF context. Second, we show that the stand-alone pod scheduling problem is NP-hard under both objectives. Third, we demonstrate that the *longest processing time first* (LPT) scheduling rule is asymptotically optimal for minimizing OCT with multiple pickers. For NRR minimization with a single picker, we provide theoretical performance bounds for the *longest/shortest processing time first* (LPT-SPT) sequencing rule. This rule alternates between pods with the largest and smallest pod processing times, balancing the workload for robots. We also introduce a modified LPT-SPT sequencing rule (M-LPT-SPT).

The computational study of our research provides three key managerial insights. First, in RMF systems where operational decisions must be made in real-time, the SPS approach is preferable to its integrated counterpart due to its efficient computation time and scalability to systems with a large number of pickers. Specifically, the sequential approach maintains a low and constant computation time regardless of the number of pickers in the system, is up to 7 times faster than the integrated approach, and achieves a mean OCT within half a percent of the integrated approach across all instances. Second, in minimizing NRR, we demonstrate that sequencing rules (LPT-SPT and M-LPT-SPT) yield similar performance when pod processing times are highly left-skewed. However, when processing times are more balanced, M-LPT-SPT outperforms LPT-SPT if shorter processing times are more common, whereas LPT-SPT is more effective when longer processing times are predominant. Our two proposed sequencing rules allow practitioners to select the best one based on their specific pod processing time distributions, reducing mean NRR by 13% to 28% without compromising order completion time, compared to the case where no sequencing rules are used. Lastly, our computational study helps managers determine the number of pickers required in an RMF system to achieve a desired mean OCT. We find that more pickers in the system decrease OCT by distributing the workload, but it also increases NRR, constituting a challenging managerial

trade-off. We show that our proposed sequencing rules help managers balance this trade-off more effectively.

Our study points to several avenues for further research. For example, like other work in the field, we consider a single RMF system. However, large e-commerce fulfillment centers might deploy multiple RMF systems that do not share system resources like robots and pods (Reindl, 2023). Designing multiple RMF systems within the same facility can be a promising research direction. Similarly, although we considered worker fatigue in our extensions, we assumed equal picking speeds for all pickers. Future research can examine the performance implications of different human picking speeds.

Acknowledgment

We thank the department editor and the review team for their constructive feedback and valuable suggestions, which significantly improved this manuscript.

Funding

The authors received no financial support for the research, authorship, and/or publication of this article.

Declaration of Conflicting Interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

ORCID iDs

Kerim U Kizil  <https://orcid.org/0000-0002-4667-3580>

Chelliah Sriskandarajah  <https://orcid.org/0000-0001-5953-3408>

Jon M Stauffer  <https://orcid.org/0000-0003-4837-459X>

Supplemental Material

Supplemental material for this article is available online (doi: 10.1177/10591478251349902).

Notes

1. <https://inviarobotics.com/blog/goods-to-person-vs-person-to-goods-automation>; <https://numinagroup.com/lp/autonomous-mobile-robots-amrs/>
2. We define an *item* as a single physical piece of stock, while *SKU* (stock keeping unit) refers to a unique identifier assigned to a specific product type or variant in the inventory. Multiple items of each SKU can be in stock, representing the available quantity in the system.
3. For example, the video linked here explains the multi-item order assembly process used in Amazon Fulfillment operations: <https://www.youtube.com/watch?v=8nKPC-WmLjU>
4. We define a *scheduling rule* as one that addresses both pod-to-picker assignment and sequencing, while a *sequencing rule* focuses only on sequencing for a picker. In the following sections, we apply the LPT scheduling rule to solve both components of the pod scheduling problem, while the LPT-SPT/M-LPT-SPT rules address only pod sequencing for a single picker.
5. We note that it might be possible for items of the same SKU to be stowed in different compartments of the same pod. In such cases, each item can have an associated setup time (say, β' , e.g., for

locating the item in the pod) in addition to the unit item picking time (α). Then, we could set $\alpha' := \alpha + \beta'$ to write $\eta_d^t = \beta + \alpha' \phi_d^t$. Then, all of our theoretical discussions remain valid when α' is substituted for α .

6. In this problem, the term *interval* refers to the intervals of real numbers, which is not to be confused with picking intervals. To make this distinction clear, we use the term *real interval* to refer to intervals in the context of MNOIP.
7. Note that with sufficiently long picker idle times, even a single robot could theoretically support a picker's schedule. However, this scenario is clearly not feasible in practice as fast order fulfillment is critical to companies and the main premise of RMF systems. Hence, we first solve the OCT minimization problem, then keep the smallest OCT achieved, and do not allow picker idle times when minimizing NRR, as in this example.
8. For example, if there are $D(t) = 50$ pods and $k = 3$ pickers, then $\rho = 2$.
9. Note that this statement implies that if $\rho = 0$, each picker will process exactly $D(t)/k$ pods in π_+^t and the optimal OCT will be $f_{OCT}(\pi_+^t) = \eta^t \times D(t)/k$ as $D(t)/k = \lceil D(t)/k \rceil = \lfloor D(t)/k \rfloor$ when $D(t)$ is divisible by k .
10. For instance, Amazon.com assigns SKUs to pods randomly (Allgor et al., 2023). While inventory allocation is beyond our paper's scope, our computational study will examine two allocation policies to demonstrate the robustness of our methods.
11. This is appropriate because, in our context, customer orders are decomposed for picking operations and later consolidated into packages at separate downstream stations (cf. Section 3). Nonetheless, we will still account for SKU correlations in our analysis—introduced through a similarity-based inventory allocation policy—by calculating how often a pair of SKUs both need to be picked within the same interval.
12. A larger expansion factor was not feasible due to rapidly increasing computation times for solving the integer programming models. However, our heuristic methods (SPS and both pod sequencing rules) scale easily to much larger systems.
13. Note that even for a single picker, the pod scheduling problem with NRR minimization objective remains NP-hard, as shown in Theorem 3.

References

- Allgor R, Cezik T and Chen D (2023) Algorithm for robotic picking in Amazon fulfillment centers enables humans and robots to work together effectively. *INFORMS Journal on Applied Analytics* 53(4): 266–282.
- Azadeh K, De Koster RBM and Roy D (2019) Robotized and automated warehouse systems: Review and recent developments. *Transportation Science* 53(4): 917–945.
- Boysen N, Briskorn D and Emde S (2017) Parts-to-picker based order processing in a rack-moving mobile robots environment. *European Journal of Operational Research* 262(2): 550–562.
- Cezik T, Graves SC and Liu AC (2022) Velocity-based stowage policy for a semiautomated fulfillment system. *Production and Operations Management* 1–16.
- Coffman EG and Sethi R (1976) A generalized bound on LPT sequencing. In: *ACM symmetrics conference*, pp.306–310. ACM Press.
- da Costa Barros ÍR and Nascimento TP (2021) Robotic mobile fulfillment systems: A survey on recent developments and research opportunities. *Robotics and Autonomous Systems* 137: 103729.
- De Koster RBM (2018) Automated and robotic warehouses: Developments and research opportunities. *Logistics and Transport* 38(2): 33.
- Garey MR and Johnson DS (1979) *Computers and Intractability: a Guide to the Theory of NP-completeness*. New York: W. H. Freeman.
- Graham RL (1969) Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics* 17(2): 416–429.
- Guizzo E (2008) Kiva systems: Three engineers, hundreds of robots, one warehouse. Available at: <https://spectrum.ieee.org/three-engineers-hundreds-of-robots-one-warehouse> (accessed 14 May 2025).
- Kim H-J, Pais C and Shen ZJM (2020) Item assignment problem in a robotic mobile fulfillment system. *IEEE Transactions on Automation Science and Engineering* 17(4): 1854–1867.
- Lamballais T, Roy D and De Koster RBM (2017) Estimating performance in a robotic mobile fulfillment system. *European Journal of Operational Research* 256(3): 976–990.
- Lamballais T, Roy D and De Koster RBM (2020) Inventory allocation in robotic mobile fulfillment systems. *IIE Transactions* 52(1): 1–17.
- Pinedo ML (2012) *Scheduling*. 4th ed. New York: Springer.
- Qin H, Xiao J, Ge D, et al. (2022) JD.com: Operations research algorithms drive intelligent warehouse robots to work. *INFORMS Journal on Applied Analytics* 52(1): 42–55.
- Rao S, Griffis SE and Goldsby TJ (2011) Failure to deliver? Linking online order fulfillment glitches with future purchase behavior. *Journal of Operations Management* 29(7–8): 692–703.
- Reindl JC (2023) Newly open Amazon fulfillment center employs 1,200+ people, mostly Detroit residents. Available at: <https://www.freep.com/story/money/business/2023/07/26/detroit-amazon-fulfillment-center-jobs/70458384007/> (accessed 14 May 2025).
- Rim    A, Gamache M, Gendreau M, et al. (2022) Robotic mobile fulfillment systems: A mathematical modelling framework for e-commerce applications. *International Journal of Production Research* 60(11): 3589–3605.
- Roy D, Nigam S, De Koster RBM, et al. (2019) Robot-storage zone assignment strategies in mobile fulfillment systems. *Transportation Research Part E: Logistics and Transportation Review* 122: 119–142.
- Shen M, Tang CS, Wu D, et al. (2020) JD.com: Transaction-level data for the 2020 MSOM data driven research challenge. *Manufacturing & Service Operations Management* 26(1): 2–10.
- Wang Z, Sheu J, Teo C, et al. (2022) Robot scheduling for mobile-rack warehouses: Human–robot coordinated order picking systems. *Production and Operations Management* 31(1): 98–116.
- Weidinger F, Boysen N and Briskorn D (2018) Storage assignment with rack-moving mobile robots in KIVA warehouses. *Transportation Science* 52(6): 1479–1495.
- Xiang X, Liu C and Miao L (2018) Storage assignment and order batching problem in Kiva mobile fulfillment system. *Engineering Optimization* 50(11): 1941–1962.

- Xie L, Thieme N, Krenzler R, et al. (2021) Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems. *European Journal of Operational Research* 288(1): 80–97.
- Yuan R, Graves SC and Cezik T (2019) Velocity-based storage assignment in semi-automated storage systems. *Production and Operations Management* 28(2): 354–373.
- Yuan R, Li J, Wang W, et al. (2021) Storage assignment optimization in robotic mobile fulfillment systems. *Complexity* 2021: 1–11.

How to cite this article

Kizil KU, Sriskandarajah C and Stauffer JM (2026) Robotic Mobile Fulfillment Systems: Strategies for Pod Selection and Scheduling. *Production and Operations Management* 35(2): 625–648.